

eBPF 기반 AI 서빙 프레임워크 취약점 가상 패치(Virtual Patching) 프레임워크 kShield-VirtualPatch

이상현^{1*}

(소속 기관)¹

kShield-VirtualPatch: An eBPF-based Virtual Patching Framework for AI Serving Framework Vulnerabilities

Sangheon Lee^{1*}

(Affiliation)¹

요약 : AI 서빙 프레임워크(Ray, vLLM, Triton 등)에서 발견되는 원격 코드 실행(RCE) 취약점은 벤더의 패치 대응이 지연되거나, 조직 내부적으로 프레임워크 업그레이드가 늦어지는 경우가 많다. 대표적으로 Ray의 Jobs Submission API에 존재하는 인증 부재 취약점(CVE-2023-48022, "ShadowRay")은 벤더가 공식 패치를 제공하지 않은 채 오랜 기간 방치되었고, 실제로 수천 대의 클러스터가 침해되어 암호화폐 채굴 등에 악용되었다. 본 논문은 eBPF를 활용하여 애플리케이션 코드를 수정하거나 재배포하지 않고도, 이러한 취약점이 실제로 악용될 때 나타나는 비정상 행위를 커널 수준에서 탐지·차단하는 가상 패치(virtual patching) 메커니즘 kShield-VirtualPatch를 제안한다. 감시 대상 AI 워커 프로세스의 계보(lineage)를 추적하여, 의심스러운 자식 프로세스 실행(SHADOW_EXEC)뿐 아니라 바이너리 종류와 무관하게 신뢰되지 않은 목적지로의 아웃바운드 연결 시도(SHADOW_CONNECT)를 "행위의 본질적 불변량" 기준으로 탐지한다. 아울러 비동기 `bpf_send_signal SIGKILL`의 잔여 한계를 보완하기 위해, LSM 혹은 시스템 콜 자체를 동기적으로 실패시키는 사전 차단 컴포넌트도 함께 구현하였다. mock Ray Jobs API 환경에서 다단계 공격 체인과 `bash /dev/tcp/` 우회 시도를 각각 실행·연결 시점에 정확히 차단하면서 정상 job은 오답 없이 통과함을 VM 실험으로 확인하였다. 비활성/SHADOW_EXEC+CONNECT/LSM 사전 차단 3개 그룹을 반복 측정한 결과 통계적으로 유의미한 차이는 없었으나(Welch's t-test), fork 집약적 워크로드에서는 작지만 유의미한 오버헤드(처리량 -0.67% , 지연시간 $+0.7\%$)가 관측되어 오버헤드가 워크로드의 fork 빈도에 비례함을 확인하였다. 같은 판정을 Falco 및 Tetragon 정책으로 옮겨 같은 VM·워크로드에서 비교한 결과, kShield-VirtualPatch만 사전에 고정된 $\pm 2\%$ 마진 안에서 동등성이 입증되었고 Falco·Tetragon은 두 워크로드 모두에서 마진을 벗어나는 유의한 오버헤드를 보였다. 독립된 커널 관측 도구(bpftrace)로 두 방어 계층이 공격을 실제로 막기까지 걸리는 시간도 별도로 측정한 결과($N=30$), LSM의 동기 차단이 v3의 비동기 SIGKILL보다 평균 $149.82\mu s$ 빨랐으며(Welch $p=2.9 \times 10^{-26}$), 그 시간 동안 v3에서는 악성 코드가 실제로 실행되었던 반면 LSM에서는 실행 자체가 없었다. 개발 과정에서 실행 파일 블록리스트의 경로·별칭 누락, 이중 용도 도구의 오답 전제, 감시 대상 프로세스 자신의 직접 행위 탐지 공백, 데몬 재시작 시 계보 유실 등 설계상 구조적 공백이 추가로 확인되어 이를 재설계·검증하였으며, 하드코딩된 rodata 및 SIEM 미연동·오답 시 세밀한 예외 부재라는 실무 도입 장벽에 대응해 신뢰 목적지 IP·감시 대상 프로세스명·의심 바이너리 목록을 재컴파일 없이 런타임에 관리하는 BPF map 기반 제어 계층과 syslog 연동, UID·cgroup 단위 예외를 추가하였다 — 이 반복 과정 자체가 행위 기반 탐지 메커니즘을 설계할 때 반드시 고려해야 할 원칙들을 보여준다.

Key Words : eBPF, Virtual Patching(가상 패치), AI Serving Framework Security(AI 서빙 프레임워크 보안), Ray, ShadowRay, CVE-2023-48022

1. 서론

AI 서빙 프레임워크(Ray, vLLM, Triton Inference Server, MLflow 등)는 대규모 LLM 추론 파이프라인의 핵심 인프라로 자리잡았다. 그러나 이러한 프레임워크들은 빠른 기능 개발 속도에 비해 보안 검토가 상대적으로 미흡한 경우가 많으며, 실제로 원격 코드 실행급 심각한 취약점이 지속적으로 보고되고 있다.

가장 널리 알려진 사례는 Ray의 Jobs Submission API에 존재하는 CVE-2023-48022("ShadowRay")이다. Ray 대시보드의 기본 포트(8265)로 노출되는 이 API는 기본 설정에서 별도의 인증을 요구하지 않아, 네트워크로 접근 가능한 누구나 임의의 코드를 클러스터의 워커 프로세스 권한으로 실행시킬 수 있다. 보안 업체 Oligo Security의 조사에 따르면 [1], 이 취약점을 통해 실제로 수천 대의 Ray 클러스터가 침해되어 암호화폐 채굴, 역방향 셸 연결, 클러스터 내 크레덴셜 탈취 등에 악용된 사례가 확인되었다. 더 심각한 문제는 벤더(Anyscale)가 이를 버그가 아니라 "Ray의 보안 경계는 Ray 클러스터 외부에 있다(Ray's security boundary is outside of the Ray cluster)"는 설계상의 결정이라는 입장을 오래 유지하며 즉각적인 공식 패치를 제공하지 않았다는 점이다 [2]. 이는 전통적인 "벤더 패치를 기다린다"는 대응 전략이 통하지 않는 상황을 만든다.

이러한 상황에서 방어자가 취할 수 있는 현실적인 선택지는 제한적이다. 프레임워크 자체를 수정하는 것은 벤더의 협조 없이는 어렵고, 프레임워크 업그레이드는 프로덕션 환경의 호환성 리스크 때문에 신중하게 이루어져야 한다. 웹 애플리케이션 보안 분야에서는 이런 상황에 대응하기 위해 WAF 수준에서 "가상 패치(virtual patching)"라는 개념이 오래전부터 활용되어 왔다 [7] — 취약점 자체를 고치지 못하는 동안, 그 취약점이 악용될 때 나타나는 특정 패턴만 탐지하여 임시로 차단하는 기법이다.

본 논문은 이 개념을 AI 서빙 프레임워크의 커널 수준으로 확장한다. 다만 이 확장에는 명확히 해 둘 범의 차이가 있다. WAF 기반 가상 패치는 악성 요청이 취약한 코드에 도달하기 전, 요청 계층에서 차단한다 — 즉 취약점 자체가 트리거되지 않는다. 반면 본 논문의 접근은 CVE-2023-48022의 인증 우회 자체를 막지 않는다: 공격자가 제출한 임의 코드는 워커 프로세스에서 실제로 실행된다. 본 논문이 "패치"라 부르는 것은, 그 실행된

코드가 이어서 시도하는 행위(자식 프로세스 생성, 외부 연결)를 커널 수준에서 가로채 실질적 피해(데이터 유출, 크립토재킹, 크레덴셜 탈취, 횡적 이동 등)로 이어지지 못하게 막는다는 의미다. 즉 "취약점의 사전 차단"이 아니라 "취약점 악용 이후 피해의 행위 수준 억제"이며, 이 구분을 2.2절에서 더 상세히 다룬다. eBPF를 이용하면 애플리케이션 코드를 한 줄도 수정하지 않고 실행 중인 프로세스에 탐지 로직을 동적으로 적재·해제할 수 있다. 기본 방어선인

SHADOW_EXEC/SHADOW_CONNECT(3.3절)는 이 무재부팅(no-restart) 성질을 배포 환경과 무관하게 유지한다. 동기적 사전 차단을 제공하는 LSM 컴포넌트(3.5절)의 사정은 다르다. 활성 LSM 스택에 bpf가 이미 포함된 배포판·커널에서는 같은 방식으로 재부팅 없이 적용되지만, 포함되어 있지 않은 구형 배포판이나 사전 설정이 안 된 노드에서는 부팅 파라미터 변경과 최초 1회 재부팅이 필요하다. 무재부팅 성질은 3.3절 방어선에는 무조건 성립하지만, 3.5절 LSM 계층에는 배포 환경에 따라 조건부로만 성립하는 셈이다. 이는 정식 패치가 배포되기 전까지의 공백을 메우는 임시 방어선이자, 정식 패치가 나온 뒤에는 혹을 제거하기만 하면 되는 저비용 대응 수단이 된다.

본 연구의 주요 기여는 다음과 같다.

- AI 서빙 프레임워크의 알려진 RCE 취약점 중 ShadowRay를 사례 연구로 삼아 eBPF 기반 가상 패치 메커니즘을 설계·구현 — 다른 프레임워크·CVE로의 일반화는 검증하지 않았으며 향후 연구로 남긴다(5장)
- 파일 접근 "빈도"가 아닌 "비정상 행위 발생 여부"를 탐지 기준으로 삼아, 단일 악용 시도로도 즉시 탐지 가능한 구조 제시
- 취약점 악용 재현(mock Ray Jobs API + exploit 스크립트)과 eBPF 방어 효과의 실증적 검증

2. 관련 연구

2.1 AI 서빙 프레임워크의 보안 취약점

Ray의 ShadowRay(CVE-2023-48022 [3][4]) 외에도, TorchServe, Triton Inference Server, MLflow 등 주요 AI 서빙 프레임워크에서 경로 탐색(path traversal), 역직렬화(deserialization) 기반 원격 코드 실행 등의 취약점이 다수 보고되었다. 이들 프레임워크는

공통적으로 (1) 빠른 기능 릴리스 주기로 인해 보안 검토가 상대적으로 후순위로 밀리고, (2) 조직 내부적으로 버전 업그레이드가 프로덕션 안정성 문제로 지연되는 경향을 보인다.

2.2 가상 패치(Virtual Patching)의 범위

가상 패치는 웹 애플리케이션 보안 분야에서 오래전부터 확립된 개념으로, ModSecurity, Cloudflare WAF, AWS WAF 등이 신규 CVE 발표 시 해당 취약점의 악용 패턴을 차단하는 임시 규칙을 신속히 배포하는 방식으로 널리 활용한다. 그러나 이러한 도구들은 대부분 HTTP 요청 계층에서 동작하며, 애플리케이션이 실제로 수행하는 시스템 수준 행위(프로세스 실행, 파일 접근 등)까지는 관측하지 못한다.

가상 패치는 문헌에서 최소 두 계층으로 논의되어 왔다.

(1) **요청 계층(request-level)**: WAF가 대표적이며, 악용 패턴을 담은 요청 자체를 취약한 코드에 도달하기 전에 차단한다 — 취약점이 트리거되는 것 자체를 막는다. (2) **행위 계층(behavior-level)**: 호스트 기반 IPS/EDR류 도구가 여기 속하며, 요청이 실제로 취약한 경로를 통과하는 것은 막지 못하지만 그 결과로 시스템에서 나타나는 이상 행위를 가로채 실질적 피해로 이어지는 것을 막는다.

본 논문의 kShield-VirtualPatch는 (2)에 속한다. CVE-2023-48022의 인증 우회는 막지 못하며, 공격자가 제출한 임의 Python 코드는 Ray 워커 프로세스 권한으로 실제 실행된다. 본 시스템이 막는 것은 그 이후 단계다 — LSM 컴포넌트(3.5절)는 시스템 콜 자체를 동기적으로 실패시키므로 해당 개별 행위는 발생 자체가 차단되지만, 이는 "실행된 악성 코드가 하려는 다음 행위"를 막는 것이지 "악성 코드가 실행되는 것" 자체를 막는 것은 아니다. SHADOW_EXEC/SHADOW_CONNECT(3.3절)는 이보다 약한 보장으로, 행위가 이미 시작된 뒤 비동기 SIGKILL로 뒤따라가 제거한다. 따라서 본 논문에서 "패치"라는 표현은 취약점을 고친 것과 동일한 코드 상태로 되돌린다는 의미가 아니라, 취약점이 악용되더라도 실질적 피해로 이어지지 않도록 막아 실무적으로 패치를 적용한 것과 동등한 위험 수준을 달성한다는 의미로 한정해서 사용한다.

2.3 eBPF 기반 런타임 보안 및 가상 패치 메커니즘

Falco [5], Tetragon [6] 등 eBPF 기반 런타임 보안 도구는 커스텀 룰을 통해 특정 시스템 콜 패턴을 탐지·차단하는 기능을 이미 범용적으로 제공한다. 즉, "eBPF로

특정 행위 패턴을 감시해서 막는다"는 메커니즘 자체는 새로운 것이 아니다. 그러나 이들 도구의 기본 룰셋은 범용 워크로드를 대상으로 하며, AI 서빙 프레임워크에 특화된 취약점에 대한 룰은 아직 정립되어 있지 않다. 본 논문의 기여는 새로운 탐지 메커니즘의 발명이 아니라, 기존 eBPF 런타임 보안 기법을 AI 서빙 프레임워크라는 특정 도메인에 적용하고 실제 CVE로 그 효과를 검증하는데 있다.

기존 도구 위에 룰을 엮는 대신 별도의 최소 구현을 처음부터 작성한 것은, 프로덕션 편의 기능을 갖추는 것보다 탐지 메커니즘 자체를 연구 목적에 맞게 외부 의존성 없이 정확히 통제된 형태로 구현·측정하는 것을 우선했기 때문이다. 이는 실용적 배포 도구로서는 Falco/Tetragon 대비 열위인 지점이다(재컴파일 없는 룰 갱신, SIEM 연동, 다중 커널 버전 검증 등은 향후 연구로 남김, 5장).

같은 판정(AI 워커 계보의 신뢰되지 않은 connect/exec)을 Falco 룰과 Tetragon 정책으로도 구성해, 같은 VM·같은 워크로드에서 오버헤드·탐지 방식을 비교했다 (4.5절). 세 도구 모두 판정 자체는 동일하게 내린다 — 그 점은 실험 설계상 당연하다. 갈리는 지점은 차단 시점과 비용이다. kShield-VirtualPatch는 두 워크로드에서 기준(도구 없음) 대비 오버헤드가 사전에 고정한 $\pm 2\%$ 마진 안에 머물렀다. Falco와 Tetragon은 두 경우 모두 마진을 벗어나는 유의한 오버헤드를 보였다 — 범용 관측 에이전트와, 좁은 판정 하나만 커널 안에서 수행하는 최소 구현 간의 비용 차이로 해석된다.

2.4 kShield(모델 파일 보안)와의 관계

본 연구는 저자의 선행 연구인 kShield(모델 파일 보안 프레임워크) [8]와 같은 eBPF kprobe/tracepoint 기반 아키텍처를 공유한다. 다만 탐지 대상과 원리가 다르다. kShield는 보안 파일에 대한 접근 **빈도**(임계값 초과)를 탐지 기준으로 삼는 반면, 본 연구는 정상적으로는 절대 발생하지 않을 **단일 행위**(비정상 프로세스 실행)의 발생 여부를 탐지 기준으로 삼는다. 이는 공격자가 파일을 여러 번 나누어 여는 대신 한 번의 `open()+read()`로 데이터를 통째로 탈취하는 경우에도 탐지가 가능하다는 구조적 이점을 가진다.

3. 설계 및 구현

기본 탐지 구조(계보 추적, SHADOW_EXEC/SHADOW_CONNECT, LSM 사전 차단, 오탐 방지, 자기 자신 직접 행위 탐지)를 확립하는 과정은 3.3~3.5절에서 서술한다. 이 기본 구조가

자리잡은 뒤로도 설계는 총 8차례 더 개선되었다 Table 1은 이를 요약한 것이다. 각 행의 상세 서술은 (3.6~3.12절). 각 개선은 예외 없이 VM 실측이나 제출 전 마지막 열에 표기된 절을 참고한다.

자체 검토 과정에서 발견된 구체적 문제에 대한 대응이며,

순번	발견한 문제	수정 내용	검증 방법	절
1차	계보 맵이 fork 후에만 의존 → 데몬이 클러스터보다 나중에 뜨거나 재시작되면 계보 누락	유저스페이스 로더가 기동 시 /proc을 스캔해 계보를 백업	2단계 손자뻘 체인으로 VM 실측	3.6
2차/3차	(a) trusted_dst_ipv4[]가 rodata라 신뢰 목적지 추가마다 재컴파일·재배포 필요 (b) SIEM 파이프라인과 단절 (c) 특정 사용자 단위 예외 불가	(a) BPF map으로 전환, kshield_ctl trust-add/del/list로 런타임 관리 (b) --syslog로 JSON 이벤트 기록 (c) exempt_uids_map으로 UID 단위 예외	6회 curl 시도 중 신뢰 등록·UID 예외 구간만 통과, journalctl JSON 대응을 VM 실측	3.7
4차	나머지 rodata 배열 잔존; UID 단위 예외만으로는 "네임스페이스" 지적의 절반만 해소	세 배열을 BPF map 전환, exempt_cgroups_map 추가, kshield_ctl 6개 리소스 체계 재구성	세 쌍 모두 즉시 반영을 VM 실측(cgroup 예외는 코드만)	3.8
5차	데몬을 kill -9로 강제 종료하면 BPF map은 판매 있어도 프로그램 부착(link)은 판매 있지 않아 커널이 즉시 회수 → fail-open. Tetragon 대조 실험에서 발견	5개 프로그램 link를 전부 bpffs에 핀. 재시작 시 새 attach를 먼저 만든 뒤 옛 핀 제거(공백 없음). SIGINT/SIGTERM에서만 명시적 unpin, kill -9는 핀이 남아 계속 보호	kill -9 후 bpftool prog list로 5개 전부 잔존·차단 지속 확인, SIGTERM으로는 핀이 실제로 사라짐도 확인	3.9
6차	watched_self[]가 fork 없는 self-exec 치환(bash -c '<단일 명령>'이 자기 자신을 execve()로 다른 바이너리로 치환)을 놓침 — post-exec 후 발동 시점엔 이미 comm이 바뀌어 있고 fork도 없어 계보 등록 안 됨. 5차 핸드오프 재측정 중 발견	execve() 진입 시점(comm 변경 전)에 새 혹은 (tp/syscalls/sys_enter_execve)을 추가해, watched_self[] 매칭 시 미리 ai_worker_lineage에 등록	파이프 없는 단일 명령 self-exec 재현(수정 전 2.008초 무탐지 → 수정 후 0.006초 즉시 차단)과 정상 job·기존 fork 케이스 회귀 없음을 VM 실측	3.10
7차	실제 Ray 클러스터 검증 중, Ray의 gRPC 클라이언트가 신뢰 등록된 목적지(GCS 자신의 IP)로도 IPv4-mapped IPv6 주소 (::ffff:a.b.c.d)로 접속을 시도해 trace_shadow_connect_v6(IPv6 전용 신뢰 목록 없음, loopback만 예외)에서 무조건 차단됨을 발견 — IPv4 신뢰 등록이 무력화됨	::ffff:0:0/96 대역을 판별해 뒤 4바이트의 실제 IPv4 주소를 기존 trusted_dst_ipv4_map으로 재판정하도록 trace_shadow_connect_v6 보강	실제 Ray 2.52.0 클러스터 (Python 3.11) 기동 후, 실제 Jobs API로 정상 job 5회 전부 SUCCEEDED, 악성 job(nc)은 SHADOW_EXEC으로 정확히 탐지·차단(exit 137)됨을 VM 실측	3.11
8차	같은 실제 Ray 클러스터에 LSM 컴포넌트 (kshield_vpatch_lsm)를 처음 붙여 검증하자, 별도 BPF 오브젝트인 kshield_lsm_socket_connect의 AF_INET6 분기에도 7차와 동일한 IPv4-mapped IPv6 신뢰 우회가 독립적으로 존재함을 발견 — 신규 위커의 GCS 연결이 매번 -EPERM으로 막혀 제출한 job 6개 전부 PENDING에 정체됨	trace_shadow_connect_v6(7차)와 동일한 판정 로직을 kshield_lsm_socket_connect에 이식	같은 실제 클러스터·실제 Jobs API로 정상 job 5회 전부 SUCCEEDED, 악성 job(nc)은 execve() 사전 차단(exit code 126, "Operation not permitted")으로 확인	3.12

Table 1. 설계 개선(1차~8차) 요약

3.1 위협 모델

본 논문이 상정하는 위협 모델은 다음과 같다. 공격자는 AI 서빙 프레임워크의 관리 API(예: Ray Jobs

Submission API, 기본 포트 8265)에 네트워크로 접근 가능하며, 해당 API에 인증 메커니즘이 부재하거나 미흡하게 설정된 취약점(CVE-2023-48022 [3][4])을 악용하여 임의의 코드를 원격에서 실행시킬 수 있다.

이 취약점은 벤더가 "의도된 설계"라는 입장을 고수하며 즉각적인 공식 패치를 제공하지 않거나 [2], 조직 내부적으로 프레임워크 업그레이드에 따른 프로덕션 호환성 리스크 때문에 패치 적용이 장기간 지연되는 상황을 전제로 한다. 실제로 Oligo Security의 조사에 따르면 [1] 수천 대의 Ray 클러스터가 이 취약점을 통해 실제로 침해되어 암호화폐 채굴, 역방향 셸 연결, 클러스터 내 자격 증명 탈취 등에 악용된 사례가 확인되었다. 이는 과거형 사례가 아니다 — 2025년 11월에도 동일한 CVE-2023-48022를 악용하는 "ShadowRay 2.0" 캠페인이 보고되었다 [10]. 공격자는 OAST 기반 정찰 서비스(interact.sh)로 인터넷 전역을 스캔해 새로 취약한 Ray 대시보드를 찾아내고, 일단 한 클러스터에 침투하면 Ray 자체의 노드 스케줄링 기능(NodeAffinitySchedulingStrategy)을 악용해 그 클러스터 안의 모든 노드로 자동 확산시킨다 — "자가 전파형"이라는 이름은 피해자의 정상 오케스트레이션 기능이 전파 도구로 무기화되었다는 뜻이지, 클러스터 사이를 감염이 건너뛰는 웜(worm) 방식이라는 뜻은 아니다. 노출된 Ray 서버는 23만 대 이상으로 집계되었다 [10].

완화책으로서의 토큰 인증과 그 한계: Ray는 2.52.0부터 선택적(opt-in) 토큰 인증(RAY_AUTH_MODE=token)을 제공한다 [11]. 이 인증이 본 논문의 위협 모델을 실제로 무력화하는지 VM에 Ray 2.52.0을 직접 설치해 검증하였다 — 인증을 켜 상태에서 `attack/exploit_shadowray.py` (토큰 없이 job 제출)를 실행하면 HTTP 401 `Unauthorized`로 요청 자체가 거부되어 코드가 실행되지 않았고, 올바른 토큰을 첨부하면 동일한 API 호출이 정상 처리됨을 확인하였다(대조군). 즉 이 인증은 본 논문이 다루는 공격 경로를 실제로 차단한다.

그럼에도 본 연구의 필요성은 유지된다. 첫째, 이 인증은 **기본값이 아니다** — Ray 개발사도 "인증은 기본값으로 전환하는 작업을 진행 중"이라고 공식적으로 밝히고 있어 [11], 현재 시점에는 기본으로 꺼져 있음을 자인한 셈이다. 둘째, 이 토큰 인증이 도입되기 전 버전에는 인증에 준하는 보호 장치조차 없었고, 그 구버전이 지금도 실제로 공격받고 있다 — 2.52.0 이전 Ray는 브라우저발 요청을 막기 위해 User-Agent 헤더가 Mozilla로 시작하지만 확인하는 약한 방식을 썼는데, 이는 DNS 리바인딩으로 우회 가능해 악성 웹페이지 방문만으로도 로컬에 떠 있는

Ray 대시보드에 셸 명령을 실행시킬 수 있었다(CVE-2025-62593). 이 취약점은 2.52.0에서 고쳐졌고 토큰 인증도 이 공격을 실제로 막지만 [11], 2026년 8월 CISA가 이를 KEV(실제 악용 취약점) 목록에 올렸다는 것은 아직 2.52.0 이전 버전을 쓰는 조직이 실제로 공격받고 있다는 뜻이다 — 본 논문이 전제하는 "패치·업그레이드가 지연되는 환경"이 인증 계층에서도 그대로 반복되고 있다는 근거다. 셋째, 가장 근본적으로 토큰 자체가 피싱이나 유출로 탈취되거나 정당한 사용자 계정이 침해되면 인증은 무력화되지만, 본 논문의 SHADOW_EXEC/SHADOW_CONNECT(3.3절)는 "공격자가 어떤 경로로 코드 실행 권한을 얻었는가"가 아니라 "그 권한으로 무엇을 하려 하는가"를 보므로, 인증 우회든 탈취된 정상 자격 증명이든 사후 행동 패턴이 같다면 동일하게 탐지·차단한다. 즉 인증과 본 논문의 행위 기반 탐지는 대체 관계가 아니라 서로 다른 계층을 담당하는 상호 보완적 방어나다.

공격자의 궁극적 목표는 원격 코드 실행을 통해 정상적인 AI 워커 프로세스의 권한으로 비정상적인 하위 프로세스를 실행하거나, 공격자가 제어하는 외부 서버로 아웃바운드 연결을 수립하여 2차 피해를 유발하는 것이다.

단, 본 논문은 ShadowRay 취약점의 원격 코드 실행 이후 악성 행위 탐지에 집중하며, 취약점 자체의 존재 여부 진단이나 초기 침투 벡터 차단은 다루지 않는다. 또한 프레임워크 소스 코드 자체를 사전에 변조하거나 커널 루트킷을 삽입하는 공격은 범위 밖으로 한다. ShadowRay 외 다른 CVE에 대한 일반화된 방어(성능·탐지율을 포함한 정식 검증)도 본 논문의 범위 밖이나, 커널 코드 변경 없이 런타임 등록만으로 다른 CVE·프레임워크도 막는지에 대한 소규모 예비 확인은 4.6절에서 수행하였다.

윤리적 고려: CVE-2023-48022는 2023년에 이미 공개되었고, 벤더가 별도 패치를 제공하지 않고 있어(위 [2]), 본 논문은 이미 공개되고 실제로 악용되고 있는 취약점에 대한 방어 기법을 다룬다. 탐지·성능 실험은 실제 Ray 클러스터가 아닌 저자가 직접 구현한 목업 서버(3.4절)만을 대상으로 하였다. 위 토큰 인증 검증만 예외적으로 실제 Ray 2.52.0을 설치해 수행하였는데, 격리된 로컬 환경(loopback 주소, 별도 가상환경)에서 파일 하나를 생성하는 무해한 명령만 제출하였고 외부로 노출되거나 실제 피해를 유발하는 행위는 없었다.

3.2 시스템 아키텍처

buffer + 사용자 공간 로거)을 따른다.

kShield-VirtualPatch는 저자의 선행 연구 kShield [8]와 동일한 아키텍처 패턴(커널 공간 BPF 프로그램 + perf

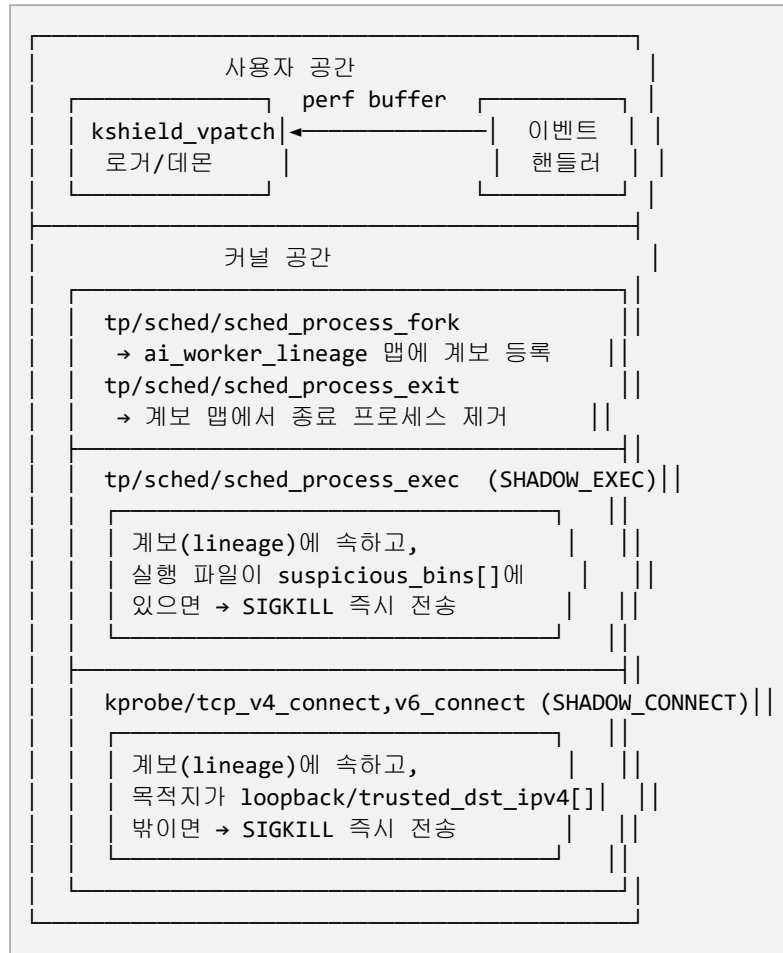


Fig. 2. kShield-VirtualPatch 기본 아키텍처

이 다이어그램은 별도의 커널 설정 없이 모든 환경에 적재 가능한 기본 방어선(kprobe/tracepoint + 비동기 SIGKILL)만을 나타낸다. 3.5절에서 별도로 다루는 실험적 LSM 컴포넌트(kshield_vpatch_lsm)는 동일한 판정 로직을 security_bprm_check_security/security_socket_connect 혹에서 동기적으로 수행하는 독립된 BPF 오브젝트이며, 위 다이어그램에는 포함되지 않는다.

3.3 핵심 탐지 메커니즘 (SHADOW_EXEC, SHADOW_CONNECT)

아래 Fig. 1은 ShadowRay 공격 체인과 그 위에 놓인 탐지 지점을 현재(1차 개선 반영) 설계로 요약한 것이다. 세부 판정 로직의 재설계 과정은 이어지는 본문에서 다룬다.

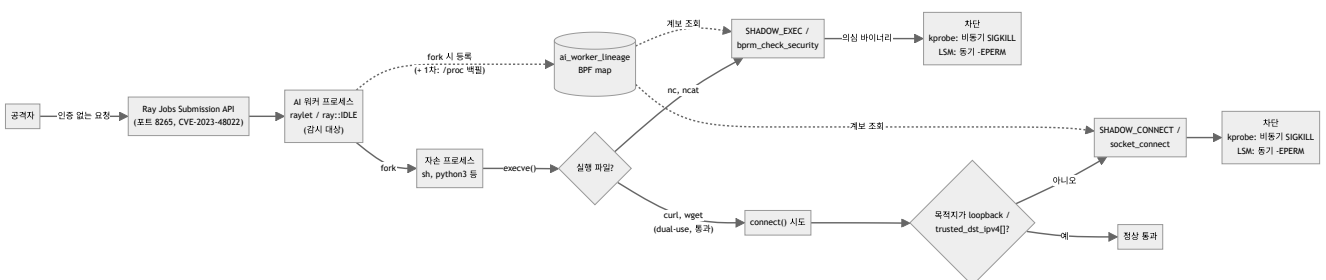


Fig. 1. 공격 체인과 탐지 지점

감시 대상 프로세스 자신이 fork 없이 직접 실행/연결하는 경우(`watched_self[]`)는 위 그림의 "AI 워커 프로세스" 노드에서 fork 단계를 거치지 않고 바로 오른쪽 경로로 진입하는 것으로 이해하면 된다.

초기 설계는 `sched_process_exec` 트레이스포인트에서 새로 실행된 프로세스의 **직속 부모만** 확인하여, 부모 comm이 `watched_parents[]`와 일치하고 실행 파일이 `suspicious_bins[]`와 일치하면 즉시 SIGKILL을 전송하는 방식이었다. 그러나 VM 실측 과정에서 두 가지 문제가 발견되었다.

1. **정상 job도 오탐:** mock Ray Jobs API는 정상/악성 job을 구분하지 않고 항상 `/bin/sh -c <entrypoint>` 형태로 실행한다. `suspicious_bins[]`에 `/bin/sh`가 포함되어 있으면 정상 job조차 실행 즉시 차단되는 문제가 실측으로 확인되었다.
2. **다단계 실행 체인 탐지 실패:** 실제 공격은 `python3(워커) → sh → curl`처럼 최소 2단계를 거친다. `curl`의 직속 부모는 `sh`이지 워커 프로세스가 아니므로, 직속 부모만 확인하는 로직으로는 탐지할 수 없었다.

이를 해결하기 위해 `sched_process_fork` 트레이스포인트를 추가로 후킹하여 **AI 워커의 자손 프로세스 계보(lineage)**를 BPF 해시맵(`ai_worker_lineage, pid → flag`)으로 추적한다. 프로세스가 fork될 때, 부모가 이미 계보에 속하거나 `watched_parents[]`와 일치하면 자식도 계보에 편입시킨다. 아울러 `/bin/sh`, `/bin/bash`는 정상 job 실행에도 쓰이는 경로이므로 `suspicious_bins[]`에서 제외하고, 실제 페이로드 다운로드·외부 연결에 쓰이는 바이너리만 남겼다.

최종 탐지 로직(`src/kshield_vpatch.bpf.c` 참조)은 다음과 같다.

1. `sched_process_fork` 시 부모가 계보에 속하거나 `watched_parents[]`와 일치하면 자식 PID를 계보에 등록
2. `sched_process_exec` 시 현재 프로세스가 계보에 속하는지(또는 직속 부모가 일치하는지) 확인
3. 계보에 속하는 경우, 실행된 바이너리가 `suspicious_bins[]`와 일치하는지 확인
4. 일치하면 이벤트를 perf buffer로 제출하고 `bpf_send_signal(SIGKILL)` 호출

이 메커니즘은 kShield의 EVIL_OPEN(반복 횟수 기반)과 근본적으로 다른 원리로 동작한다. AI 워커 계보에서 `curl/wget` 등이 실행되는 행위는 정상 운영 중에는 **단 한**

번도 발생해서는 안 되는 행위이므로, 반복 횟수와 무관하게 첫 발생 시점에 즉시 탐지·차단할 수 있다.

`watched_parents[]`와 `suspicious_bins[]`는 BPF rodata 섹션에 컴파일 타임에 삽입되며, 이는 kShield [8]와 동일하게 대상 경로 변경 시 재컴파일이 필요하다는 한계를 가진다(3.7, 3.8절에서 해소). 전체 구현 코드는 공개 저장소 [9]에서 확인할 수 있다.

SHADOW_CONNECT — 실행 파일 블록리스트의 근본적 우회 문제 해결. SHADOW_EXEC는 실행 파일 이름 블록리스트에 의존하는데, 다음과 같이 쉽게 우회 가능하다는 한계가 있었다. (1) `bash` 내장 기능 우회: `bash -c 'exec 3<>/dev/tcp/host/port; ...'`은 별도 바이너리를 실행하지 않는다. (2) `watched_parents`의 `python3`가 소켓 API로 직접 통신하면 실행 파일 이름 기반 검사를 통과한다. (3) 별도 프로세스 실행 없이 직접 통신하면 애초에 감시할 `exec()` 이벤트 자체가 없다.

이 세 우회의 공통점은 결국 커널의 `tcp_v4_connect/tcp_v6_connect`를 거쳐 아웃바운드 TCP 연결을 시도한다는 점이다. 이 지점을 직접 후킹하여, "어떤 바이너리를 실행했는가"가 아니라 **"AI 워커 계보에서 신뢰되지 않은 목적지로 연결을 시도했는가"**를 감시한다. 기존 SHADOW_EXEC은 그대로 유지하여 두 계층이 함께 방어한다(defense-in-depth) — 알려진 바이너리는 실행 시점에 더 일찍 잡고, 그 외 모든 경로는 연결 시도 시점에 잡는다.

위 우회 (2)는 흔히 "living-off-the-land"(공격자가 별도 도구를 반입하지 않고 대상 시스템에 이미 있는 정상 기능만으로 공격하는 기법)로 분류되는 유형에 해당한다. SHADOW_CONNECT는 "어떤 프로그램이 연결을 시도했는가"가 아니라 "커널의 `connect` 경로를 거쳐 어디로 연결을 시도했는가"만 판정하므로, `curl`이든 파이썬의 `socket` 모듈이든 결과는 동일하며 이 유형의 우회는 이 설계 자체로 대응된다. 반면 새 연결을 맺지 않고 이미 신뢰받아 열려 있는 기존 연결 위에 데이터를 얹어 유출하는 경우는 이 메커니즘이 감시하는 지점(`connect()` 호출)이 발생하지 않으므로 탐지되지 않는다.

아울러 초기에는 프로세스 종료 시 `ai_worker_lineage` 맵을 정리하는 로직이 없어, PID가 재사용될 경우 무관한 새 프로세스가 죽은 프로세스의 계보 정보를 잘못 물려받을 수 있는 버그가 있었다. `sched_process_exit` 훅으로 프로세스 종료 시 계보 정보를 제거하여 이를 해결하였다.

SHADOW_CONNECT의 한계: SHADOW_CONNECT도 완전한 차단은 아니다. (1) 이미

열려 있는 정상 연결에 얹혀 데이터를 빼가는 경우, (2) 허용된 포트/프로토콜 위로 데이터를 숨기는 터널링, (3) 네트워크가 아예 필요 없는 로컬 전용 공격은 이 메커니즘으로 포착되지 않는다. 또한 `bpf_send_signal` 은 비동기적이므로 완전한 사전 차단을 수학적으로 보장하지는 않는다 — 이 한계를 해소하기 위해 LSM 혹은 기반의 동기적 사전 차단을 별도 컴포넌트로 구현하였다 (3.5절).

오탐 방지: curl/wget의 이중 용도 문제. `suspicious_bins[]` 는 curl/wget 실행 자체를 목적지와 무관하게 즉시 이상 행위로 간주하지 않는다. 실제 AI 서버 워크로드에서는 워커가 모델 가중치·데이터셋을 curl/wget으로 내려받는 것이 정상적인 운영이기 때문이다 — "curl을 실행했는가"는 이상 행위의 신호가 아니고, "curl이 어디로 연결하는가"가 실제 신호다. curl/wget은 `suspicious_bins[]` 에서 제외하고 판단을 SHADOW_CONNECT(및 LSM `socket_connect`)에게 전적으로 넘긴다 — exec 시점에는 통과시키고, 실제 connect() 시도 시점에 목적지가 신뢰되지 않으면 그때 잡는다. nc/ncat은 AI 워커 계보에서 합법적으로 실행될 이유가 사실상 없어 목적지와 무관하게 exec 즉시 차단을 유지한다.

아울러 ModSecurity, Cloudflare WAF 등 가상 패치의 원조 도구들은 신규 룰을 배포할 때 "탐지만 하고 차단은 안 함"(alert-only/dry-run) 단계를 거쳐 오탐을 관찰한 뒤에야 실제 차단으로 전환한다(2.2절). 이 단계적 배포를 지원하는 `enforce_mode` 전역 변수를 두었다 — 재컴파일 없이 `--audit-only` 플래그로 값을 설정할 수 있다. VM 실측 결과, 두 시나리오 모두에서 SHADOW_EXEC/LSM_EXEC_BLOCK은 더 이상 curl에 발동하지 않았고, 신뢰 목적지 다운로드도 완전히 정상 처리되었으며, 신뢰 안 된 목적지로의 시도는 exec 대신 connect 계층에서 여전히 정확히 차단됨을 확인하였다.

감시 대상 프로세스 자신의 직접 행위 탐지. `current_is_watched()` 의 판정 조건은 둘 다 "자손 프로세스"만 커버한다는 공백이 있었다 — 감시 대상 프로세스 본인(예: raylet)이 fork 없이 직접 `execve()`나 `connect()`를 호출하면 어느 조건도 참이 되지 않는다. 이를 메우기 위해 `watched_parents[]`와 별도로 `watched_self[]` (raylet, ray::IDLE만 포함)를 두고, "내 자신의 comm이 `watched_self[]`와 일치하는가"를 세 번째 조건으로 추가하였다. VM 실측 결과, /bin/nc를 파일명만 raylet로 복사해 fork 없이 직접 실행하자 SHADOW_CONNECT/LSM_CONNECT_BLOCK

양쪽에서 정확히 잡혔고, 이름을 바꾸지 않은 평범한 nc는 여전히 아무 영향을 받지 않아 오탐 없음을 확인하였다. 이 검사는 `watched_self[]`에 등록된 이름과 정확히 일치하는 comm에만 적용되며, 프로세스명 위장이나 인프로세스 코드 인젝션은 다루지 않는다(5장).

3.4 공격 재현 환경

실제 Ray 클러스터를 설치하지 않고도 ShadowRay의 핵심 취약점(인증 없는 원격 코드 실행)을 재현하기 위해 목업 Jobs API 서버(attack/mock_ray_server.py)를 구현하였다. 이 서버는 /api/jobs/ 엔드포인트로 제출된 `entrypoint` 명령어를 인증 검사 없이 그대로 서브프로세스로 실행하여, 실제 Ray Jobs API의 핵심 취약 동작을 그대로 재현한다.

3.5 LSM 혹은 기반 동기적 사전 차단

SHADOW_EXEC/SHADOW_CONNECT는 모두 "행위가 발생한 뒤 `bpf_send_signal(9)`로 프로세스를 죽이는" 방식이다. `bpf_send_signal`은 비동기 전달이므로, 신호가 실제로 전달되기 전에 connect()가 이미 진행되었거나 `execve()`가 이미 완료되었을 여지를 이론적으로 완전히 배제하지 못한다. 이를 근본적으로 해소하려면 시스템 콜의 반환값 자체를 커널이 원래 진행을 이어가기 전에 대체해야 한다. `BPF_PROG_TYPE_LSM` 프로그램은 LSM 혹은 지점에 `fmod_ret` 트램폴린 방식으로 부착되어, 원래 함수가 반환하기 전에 그 반환값 자체를 대체할 수 있다. 이 잔여 시간차가 실제로 몇 마이크로초인지를 독립된 커널 관측 도구로 정량화한 결과는 4.7절에서 다룬다.

- `security_bprm_check_security` 혹에서 AI 워커 계보가 `suspicious_bins[]`를 실행하려 하면 -EPERM을 반환하여 `execve()` 자체를 실패시킨다.
- `security_socket_connect` 혹에서 신뢰되지 않은 목적지로 `connect()`를 시도하면 -EPERM을 반환하여 연결 자체를 실패시킨다.

`BPF_PROG_TYPE_LSM`은 `CONFIG_BPF_LSM=y` 커널과, 활성 LSM 목록에 "bpf" 포함을 요구한다. 실험 환경(4.1절)에서는 부팅 파라미터에 `lsm=...,bpf`를 추가하고 재부팅한 뒤 attach가 성공하였다. 이 재부팅 요구사항 자체가 실무적 배포 비용이다 — 특히 관리형 쿠버네티스(EKS, GKE 등)나 여러 팀이 공유하는 GPU 클러스터에서는 노드 하나를 재부팅하는 것이 애플리케이션 재시작보다 훨씬 큰 작업이므로, 재부팅 가능 여부를 사전에 협의해야 한다. 이 요구사항이

SHADOW_EXEC/SHADOW_CONNECT(3.3절)에는 없다는 점이 본 논문이 두 계층을 모두 유지하는 이유다.

VM 실행 결과: bpf tool link list로 확인한 결과 두 혹은 모두 attach_type lsm_mac 으로 동일하게 attach되었다.

```
[정상 job] echo benign-job → 로그 없음, 오탐 없이 정상 처리
[블록리스트 우회] bash -c 'exec 3<>/dev/tcp/1.1.1.1/80; ...' →
    LSM_CONNECT_BLOCK 사전 차단! proc=bash parent=sh dst=1.1.1.1:80 => connect() -EPERM
    (mock 서버 측: "bash: connect: Operation not permitted")
```

실행 중 발견한 실제 버그: 알려진 바이너리(curl) 실행 테스트에서 처음에는 LSM_EXEC_BLOCK 로그가 찍혔음에도 불구하고, 같은 PID가 바로 이어서 comm=curl로 connect()를 시도하는 LSM_CONNECT_BLOCK 이벤트가 발생하는 모순이 관측되었다. bpf_printk로 커널 트레이스를 직접 확인한 결과, bprm_check_security 혹은 execve("/usr/bin/curl", ...)를 -EPERM으로 거부하자, 셸의 \$PATH 탐색이 같은 파일을 가리키는 다른 경로 문자열인 /bin/curl로 재시도하였고, 이 경로는 suspicious_bins[]에 없어 통과되었다. 두 경로를 모두 등록하여 수정하였으며, 이 배열은 v3(3.3절)와 동일하므로 같은 누락이 이미 검증된 v3 결과에도 존재했었다 — 재검증하여 함께 수정하였다.

수정 전 이 버그가 존재하던 상태에서도 실제로 데이터가 외부로 유출되지는 않았다 — curl이 우회 실행에 성공한 직후 시도한 connect()가 security_socket_connect 계층에서 다시 차단되었기 때문이다. 이는 defense-in-depth 구조가 실제로 작동함을 보여주는 사례다.

검증 상태: 빌드·attach 성공, execve()/connect() 사전 차단 동작(수정 후), 정상 job 오탐 없음을 VM에서 확인하였다. 성능 오버헤드도 4.4절에서 v3 대비 측정하여 유의미한 차이가 없음을 확인하였다(처리량 p=0.6228, 지연시간 p=0.9348). 다만 탐지 동작 확인은 여전히 단일 실행 기반 정성적 확인이며, N회 반복 탐지율 등에 대한 검증은 아직 수행하지 않았다(5장).

3.6 1차 — 데몬 재시작·최초 기동 시 계보 유실

발견 경위: ai_worker_lineage 맵이 오직 sched_process_fork 혹은 발동하는 순간에만 채워진다는 한계가 확인되었다. 데몬이 이미 실행 중인 AI 워커 클러스터에 나중에 배포되거나 재시작되는 경우, 데몬이 기동되기 전부터 존재하던 워커의 자손 프로세스는 계보에서 누락된다.

수정: 커널 BPF 프로그램은 건드리지 않고, 유저스페이스 로더가 attach 직후 1회 /proc 을 스캔하여 이미 실행 중인 프로세스들의 조상(ancestor) 체인을 확인하고, 감시 대상과 일치하면 동일한 ai_worker_lineage 맵에 직접 채워 넣는다.

VM 실행 결과: 데몬을 켜기 전에 미리 raylet → bash → (20초 뒤) curl로 이어지는 2단계 자손 체인을 백그라운드로 띄워 둔 뒤, 그 상태에서 데몬을 새로 기동하여 백필 로그와 이후 탐지 여부를 확인하였다.

```
$ sudo ./kshield_vpatch
계보 백필: 데몬 기동 전부터 이미 실행 중이던 AI 워커 계보 프로세스 4개를 편집함
...
[04:46:17] SHADOW_CONNECT 탐지! parent=bash -> proc=curl(pid=820412) dst=1.1.1.1:80 => SIGKILL 전송

$ sudo ./kshield_vpatch_lsm
계보 백필: ... N개를 편집함
[04:49:22] LSM_CONNECT_BLOCK 탐지! proc=curl(pid=820412) parent=bash dst=1.1.1.1:80 => connect() -EPERM
```

curl의 직속 부모가 bash인데도(직속 부모 검사로는 설명되지 않는 데도) 정확히 잡혔다는 것은, 데몬 기동 전에 이미 떠 있던 bash가 백필로 계보에 편집되었고 그 bash가 이후 fork한 curl 자식이 계보를 자동으로 물려받았다는 뜻이다.

남는 한계: /proc 스캔과 맵 갱신 사이에는 이론적으로 아주 좁은 경합 창이 있다. 다만 이는 기존 PID 재사용 리스크(3.3절)와 같은 계열이며, 그 프로세스가 종료되면 동일한 exit 혹은 정리된다.

3.7 2차/3차 — 운영 장벽 해소: 런타임 제어 계층과 SIEM 연동

발견 경위: 실무 도입 관점에서 다음 세 가지 한계가 확인되었다. (1) trusted_dst_ipv4[] 가 컴파일 타임 rodata 배열이라 재컴파일·재배포가 필요하여 SecOps 팀이 도입할 유인이 낮다. (2) 이벤트가 perf buffer/표준출력에만 남아 기존 SIEM 파이프라인과 단절되어 있다. (3) 오탐 시 GPU 비용이 매우 큰데도 세밀한 예외 제어가 불가능하다.

수정: trusted_dst_ipv4_map(BPF 해시맵)으로 전환하고 /sys/fs/bpf/에 편하여 kshield_ctl 유틸리티가 재시작 없이 add/del/list하도록 하였다. 두 데몬 모두에 --syslog 플래그를 추가하여 탐지 이벤트를 syslog(LOG_AUTHPRIV)에 JSON 한 줄씩 남기도록 하였다(로그 인젝션 방지를 위해 공격자 통제 값은 이스케이프). exempt_uids_map을 추가하여 UID 단위

감시 예외를 `current_is_watched()` 의 최우선 조건으로 확인하도록 하였다.

VM 실행 결과: mock Ray Jobs API로 신뢰 안 된 목적지(1.1.1.1)로의 curl job을 6회 제출하고, 그 사이에 `trust-add/del`, `exempt-add/del`을 병행하여 재현하였다.

```
[시도1, 예외 없음] → 차단(Killed), SHADOW_CONNECT 탐지
$ sudo ./kshield_ctl trust-add 1.1.1.1
[시도2, IP 신뢰 등록 후] → 통과(로그 없음)
$ sudo ./kshield_ctl trust-del 1.1.1.1
[시도3, IP 신뢰 해제 후] → 차단(Killed)
[시도4, UID 예외 없음] → 차단(Killed)
$ sudo ./kshield_ctl exempt-add 1009
[시도5, UID 예외 등록 후] → 통과(로그 없음)
$ sudo ./kshield_ctl exempt-del 1009
[시도6, UID 예외 해제 후] → 차단(Killed)

$ sudo journalctl -t kshield_vpatch -n 20 --no-pager
{"tool": "kshield_vpatch", "event": "SHADOW_CONNECT", "parent_comm": "sh", "comm": "curl", "pid": 838948, "dst_ip": "1.1.1.1", "dst_port": 80, "enforced": true}
... (총 4건, 시도 1/3/4/6과 정확히 대응)
```

6회 시도 중 예외가 적용된 2회만 정확히 통과했고, 나머지 4회는 모두 차단되어 `journalctl`에 JSON 4건으로 정확히 대응됨을 확인하였다 — 등록·해제 모두 재시작·재검파일 없이 즉시 반영됨을 검증하였다.

남는 한계: (1) `watched_parents[]/suspicious_bins[]` 는 여전히 `rodata`로 남아 있다(→ 3.8절에서 해소). (2) 예외는 UID 단위까지만 지원한다(→ `cgroup` 단위는 3.8절에서 부분 해소). (3) `syslog` 연동은 로컬 소켓 도달까지만 확인하였고, 실제 SIEM까지의 도달·파싱은 검증하지 않았다.

3.8 4차 — 남은 rodata 들의 런타임화와 cgroup 단위 예외

발견 경위: 3.7절의 개선은 신뢰 목적지 IP에 한해서만 이루어졌다. "나머지 배열은 IP만큼 자주 바뀌지 않는다"는 전제는 실무적 근거가 충분하지 않은 것으로 판단된다.

수정: `watched_parents[]/watched_self[]/suspicious_bins[]`를 동일한 방식으로 BPF 해시맵으로 전환하였다. 맵이 새로 생성될 때만 기존 기본값을 시드로 채워, 운영자가 `kshield_ctl`로 바뀌든 값이 재시작 때마다 되돌아가지 않는다. `exempt_cgroups_map`을 추가하여 UID보다 세밀한 컨테이너/파드 단위 예외를 지원한다 — 다만 쿠버네티스 "네임스페이스" 자체는 커널이 아는 개념이 아니므로, "네임스페이스 인지"라고

과장하지 않고 "`cgroup`(≈컨테이너/파드) 단위"로 정확히 이름 붙였다.

VM 실행 결과: 데몬을 최초 기동한 뒤, 다음을 순서대로 확인하였다.

```
$ sudo ./kshield_ctl parent-add myworker
$ cp /bin/bash /tmp/myworker && /tmp/myworker -c 'curl http://1.1.1.1/ | cat' &
[18:18:45] SHADOW_CONNECT 탐지! parent=myworker -> proc=curl dst=1.1.1.1:80 => SIGKILL 전송
$ sudo ./kshield_ctl parent-del myworker
$ /tmp/myworker -c 'curl http://1.1.1.1/ | cat' & → 로그 없음(정상 통과)

$ sudo ./kshield_ctl bin-add /usr/bin/telnet
$ telnet 1.1.1.1 80 → SHADOW_EXEC 탐지! exec=/usr/bin/telnet, Killed
$ sudo ./kshield_ctl bin-del /usr/bin/telnet
$ telnet 1.1.1.1 80 → SHADOW_EXEC은 안 뜨지만 SHADOW_CONNECT 탐지! parent=sh -> proc=telnet dst=1.1.1.1:80 => SIGKILL 전송

$ sudo ./kshield_ctl self-add myworker2
$ cp /usr/bin/curl /tmp/myworker2 && timeout 5 /tmp/myworker2 http://1.1.1.1/
[18:55:12] SHADOW_CONNECT 탐지! => SIGKILL 전송 (종료 코드: 137)
$ sudo ./kshield_ctl self-del myworker2
$ timeout 5 /tmp/myworker2 http://1.1.1.1/ → 종료 코드: 124 (안 잡힘, 정상)
```

`parent-add/del`, `bin-add/del`, `self-add/del` 세 쌍 모두 데몬 재시작·재검파일 없이 즉시 반영됨을 확인하였다. 특히 `bin-del` 이후에도 `SHADOW_CONNECT`가 대신 탐지한 결과는 `defense-in-depth` 설계를 뒷받침한다.

cgroup-exempt-*의 VM 검증(추가 라운드): 처음 구현 당시에는 `exempt_uids_map`과 동일한 코드 경로(핀 → 조회 → `current_is_watched()` 최우선 확인)를 공유한다는 점에만 근거해 VM 검증을 생략하였다. 이번에 실제 `cgroup v2` 환경에서 직접 확인하였다 — 테스트용

`cgroup(/sys/fs/cgroup/kshield_test_cgroup, cgroup ID는 해당 디렉터리의 inode 번호로 확인 가능)을 만들고, 그 cgroup 안에서 실행한 공격(nc, fork 강제 재현)이 예외 등록 전에는 0.007초 만에 차단되었다. kshield_ctl cgroup-exempt-add <cgroup_id>로 등록한 뒤 같은 cgroup에서 재현하자 차단되지 않고 nc 자신의 타임아웃(2초, 실효 2.010초)까지 그대로 실행되었고, cgroup-exempt-del로 삭제한 뒤에는 다시 0.006초 만에 차단됨을 확인하였다 — exempt_uids_map과 동일한 신뢰도로 동작함을 실효로 확정하였다.`

남는 한계: (1) 실제 쿠버네티스 네임스페이스 단위 예외는 여전히 근본적으로 불가능하다 — cgroup ID는 근사치일 뿐이며, K8s API를 감시하는 별도 컨트롤 플레인이 필요하다(범위 밖). (2) syslog의 SIEM 종단 도달 검증은 여전히 미해결이다. (3) kshield_ctl의 하위 명령이 18개(6개 리소스 × 3개 동작)로 늘어나, 벌크 설정 기능은 아직 없다.

3.9 5차 — 데몬 크래시 시 fail-open 문제와 BPF link 핀

발견 경위: 4.5절에서 이미 Falco·Tetragon과 오버헤드·탐지 방식을 비교했지만, "데몬 프로세스 자체가 죽으면 어떻게 되는가"는 비교한 적이 없었다. 이를 확인하기 위해 kShield-VirtualPatch와 Tetragon 양쪽에서 데몬을 kill -9(정상 종료는 아닌, 크래시를 흉내 낸 강제 종료)로 죽인 뒤 커널에 BPF 프로그램이 남아있는지, 같은 공격이 여전히 차단되는지를 VM에서 직접 확인하였다. kShield-VirtualPatch는 kill -9 직후 bpftool prog list에서 trace_shadow_exec/trace_shadow_connect_v4/trace_shadow_connect_v6 전부가 사라졌다 — 데몬과 함께 커널의 탐지 로직 자체가 통째로 사라지는 **fail-open**이었다. 반면 Tetragon은 kill -9 후에도 generic_kprobe_* 프로그램 7개가 그대로 남아 있었는데, 우연히 그 시점에 systemd가 이미 재시작시킨 결과가 아닌지 확인하기 위해 재시작 지연(RestartUsec=5s)이 지나기 전인 첫 2초 동안 200ms 간격으로 MainPID와 BPF 프로그램 수를 함께 추적하였다 — 10회 샘플 전부 MainPID=0(데몬이 완전히 죽어 있음)이면서 BPF 프로그램 수는 7개로 그대로였다. 즉 Tetragon은 데몬이 실제로 완전히 죽어 있는 동안에도 커널 쪽 판정·차단이 데몬과 무관하게 계속 동작하고, kShield-VirtualPatch는 그렇지 않았다.

원인은 2차~4차에서 trusted_dst_ipv4_map 등 맵은 bpffs에 핀했지만, 실제 판정·SIGKILL을 수행하는 BPF 프로그램의 부착(attach, struct bpf_link) 자체는 한 번도 핀한 적이 없었다는 것이다. 데몬 프로세스가 그 attach를 가리키는 파일 디스크립터를 들고 있다가, 프로세스가 죽으며 fd가 함께 닫히면 커널이 참조 카운트 0을 보고 프로그램을 자동으로 회수한다.

수정: 다섯 개 프로그램(trace_lineage_fork, trace_lineage_exit, trace_shadow_exec, trace_shadow_connect_v4, trace_shadow_connect_v6)의 link를 attach 직후 bpffs에 핀한다. 재시작 시 이전 실행(크래시)이 남긴 핀이 있으면, 새 attach를 먼저 만든 뒤에 옛 핀을 지운다

— 이 순서라면 옛 attach가 사라지는 시점엔 이미 새 attach가 똑같이 동작 중이라 보호 공백이 생기지 않는다. 다만 이 모든 게 "데몬을 다시 띄워야" 일어나는 일이므로, kShield-VirtualPatch 자체는 크래시 후 자동 재시작(systemd Restart=on-failure 같은) 기능이 없어 이 경로가 실제로 언제 실행될지는 운영자가 데몬을 다시 띄우는 시점에 달려 있다 — 이번 수정이 보장하는 것은 "재시작 전까지도 커널이 계속 막아준다"는 것이지, "자동으로 재시작된다"는 것은 아니다. 반대로 SIGINT/SIGTERM 같은 의도된 종료에서는 운영자가 실제로 보호를 끄고 싶은 것이므로, 종료 직전에 핀을 명시적으로 지운다. 지금까지는 SIGINT만 잡았는데, kill <pid>나 systemctl stop의 기본 시그널은 SIGTERM이라 이번에 함께 잡도록 추가하였다. kill -9(SIGKILL)는 애초에 어떤 핸들러도 타지 않으므로 이 unpin 경로를 거치지 않고, 그래서 핀이 그대로 남아 계속 보호한다 — 의도한 동작이다.

VM 실험 결과: 수정 후 재빌드하여 같은 시나리오를 재현하였다.

```
[데몬 기동] /sys/fs/bpf/kshield_link_{lineage_fork,lineage_exit,shadow_exec,shadow_connect_v4,shadow_connect_v6} 5개 핀 파일 생성 확인

[kill -9] $ sudo kill -9 $KSPID
$ sudo bpftool prog list | grep -A1 'shadow\|lineage'
→ 5개 프로그램(trace_lineage_fork/exit, trace_shadow_exec, trace_shadow_connect_v4/v6) 전부 잔존 확인

[데몬 없는 상태에서 공격 재현] nc -w 2 203.0.113.1 80
→ nc가 자신의 2초 타임아웃보다 먼저 사라짐(1초 이내) - 커널에 남은 프로그램이 여전히 SIGKILL을 보내고 있음을 시사

[SIGTERM으로 정상 종료] $ sudo kill $KSPID2 (kill -9 아님)
→ 종료 로그: 정상 종료 메시지("Done", "Killed" 아님)
$ sudo ls /sys/fs/bpf/ | grep kshield_link
→ 핀 파일 전부 사라짐(정상 종료 시 실제로 보호가 꺼짐을 확인)
```

즉 (1) 크래시(kill -9)에서는 커널에 남은 프로그램이 계속 보호하고, (2) 의도된 종료(SIGTERM)에서는 실제로 보호가 꺼진다는 두 요구사항 모두 VM에서 확인하였다.

kshield_vpatch_lsm(3.5절)으로 확장: 이 발견이 SIGKILL 기반 컴포넌트만의 문제인지, 동기적 사전 차단(LSM) 컴포넌트에도 같은 문제가 있는지 이어서 확인하였다. 강제 종료 전 execve(nc)는 0.008초 만에 -EPERM(종료 코드 126)으로 즉시 실패했으나, kill -9

직후 bpftool prog list에서 네 프로그램 (trace_lineage_fork/exit, kshield_lsm_bprm_check, kshield_lsm_socket_connect) 전부가 사라졌고, 같은 시도가 이번에는 2.010초 동안 실제로 실행되어(nc 자신의 -w 2 타임아웃과 일치) fail-open이 그대로 재현되었다. kshield_vpatch.c와 동일한 수정(네 개 link를 bpffs에 핀, 재시작 시 새 attach 우선 생성, SIGINT/SIGTERM에서만 명시적 unpin)을 적용한 뒤 재검증한 결과, kill -9 후에도 네 프로그램이 전부 잔존하며 execve(nc)가 여전히 0.006초 만에 즉시 차단되었고, SIGTERM으로는 핀이 실제로 사라짐(정상 종료 시 보호가 꺼짐)도 확인하였다 — 두 컴포넌트 모두 같은 근본 원인-같은 수정으로 해소되었다.

재시작 핸드오프 구간의 밀리초 단위 재측정: 위 검증은 "데몬이 재시작 없이 계속 죽어 있는 동안"만 다루었을 뿐, 실제로 재시작이 일어나는 순간 자체는 겨냥하지 않았다. 이를 보완하기 위해 kshield_vpatch_lsm과 kshield_vpatch(kprobe/SIGKILL 기반) 양쪽에서, kill -9 후 2초 대기(위와 동일하게 커널에 남은 프로그램이 계속 차단함을 재확인)한 다음 데몬을 수동으로 재기동함과 동시에 공격(watched_parents[]에 등록된 이름으로 위장한 bash, 즉 3.6절과 동일한 /tmp/raylet가 자식으로 nc를 실행)을 간격 없이 150회 연속 재현하여 각 시도의 소요시간을 기록하였다. kshield_vpatch_lsm은 150회 전부 0.004~0.005초 내로 즉시 차단(execve() -EPERM, 종료 코드 126) 되었고 표본 구간은 약 2.82초였다. kshield_vpatch(SIGKILL)도 동일하게 150회 전부 0.006~0.008초 내로 즉시 차단되었고 표본 구간은 약 3.10초로, 두 구간 모두 데몬 재기동(BPF 스케레톤 로드·attach·핀 재생성) 과정 전체를 포괄한다. 재시작된 각 데몬 자신의 로그에도 150회 각각이 정상적으로 탐지·기록되어 있었고, 재기동 후 핀 파일이 모두 정상적으로 재생성된 것도 함께 확인하였다.

이 재측정 과정에서 5차와는 무관한 별개의 방법론적 함정을 하나 발견하였다. kshield_vpatch(kprobe) 재현에 처음에는 /tmp/raylet -c "nc -w 2 <IP> 80"처럼 단일 명령을 그대로 썼는데, 이 경우 bash가 fork 없이 자기 자신을 execve()로 nc로 치환하는 최적화를 수행한다. tp/sched/sched_process_exec 혹은 execve() 완료 이후에 발동하므로 그 시점엔 이미 comm이 raylet에서 nc로 바뀌어 있어 watched_self[] (3.3절)에 매칭되지 않고, fork가 없었으니 ai_worker_lineage에도 등록되지 않아 탐지 자체가

발생하지 않았다(150회 전부 2.008초 동안 실제로 실행되어 탐지 실패로 나타남). 반면 kshield_vpatch_lsm의 bprm_check_security는 execve() 이전에 발동하므로 그 시점엔 comm이 아직 raylet라 watched_self[] 매칭이 그대로 성공한다 — 같은 코드(watched_self[] 검사)가 혹의 실행 시점 차이만으로 kprobe 쪽에서만 무력화된 것이다. 3.8절에서 쓴 것과 같이 파이프(| cat)를 추가해 bash가 nc를 별도 자식으로 fork하도록 강제하자(→ ai_worker_lineage 경로로 정상 등록·탐지), 두 컴포넌트 모두 동일하게 즉시 차단되었다 — 이후 보고한 150회 결과는 이 fork 강제 재현으로 얻은 것이다.

제어 평면(kshield_ctl)의 데몬 독립성 확인: 5차의 핵심 주장은 "데몬이 죽어도 커널의 판정·차단 로직은 계속 동작한다"는 것이었다. 이 주장의 자연스러운 확장으로, 데몬이 완전히 죽어 있는 동안 운영자가 kshield_ctl로 설정을 바꾸는 것도 가능한지, 그리고 그 변경이 죽어 있는 데몬과 무관하게 즉시 반영되는지를 확인하였다. kshield_vpatch를 정상 기동한 뒤 kill -9로 강제 종료하고 데몬이 완전히 죽어 있음을 pgrep으로 재확인한 상태에서, 신뢰 안 된 목적지(203.0.113.1)로의 curl 공격 재현(exec에는 안 걸리고 connect 계층에서만 걸리는 경로)이 0.017초 만에 차단됨을 먼저 확인하였다. 이 상태 그대로 sudo ./kshield_ctl trust-add 203.0.113.1을 실행하자 명령이 정상 완료되고 trust-list에도 즉시 반영되었으며, 같은 공격을 재현하자 이번에는 차단되지 않고 curl 자신의 타임아웃(2초, 실측 2.018초)까지 그대로 실행되었다 — 데몬 프로세스가 존재하지 않는 상태에서 방금 추가한 신뢰 설정이 커널의 판정에 즉시 반영된 것이다. 이어서 같은 상태에서 trust-del로 되돌리자 trust-list도 즉시 비워졌고, 재현 공격도 다시 0.016초 만에 차단되어 원상복구까지 확인하였다. 전 과정(추가→통과 확인→삭제→차단 확인)에서 데몬 프로세스는 단 한 번도 재기동되지 않았다(pgrep으로 각 단계마다 재확인). 이는 2차~4차에서 맵을 bpffs에 핀한 설계와 5차에서 프로그램 link까지 핀한 설계가 결합하여, 제어 평면(kshield_ctl)과 데이터 평면(커널의 판정 로직) 모두 유저스페이스 데몬 프로세스의 생존 여부와 완전히 독립적으로 동작함을 보여준다.

남는 한계: (1) 호스트 자체가 재부팅되는 경우는 다루지 않는다 — bpffs는 보통 메모리 기반이라 재부팅되면 핀도 함께 사라지므로, 이 수정이 보장하는 것은 "데몬만 크래시하는" 상황에 한정된다(두 컴포넌트 공통). (2) 재시작 핸드오프 구간은 위 재측정으로 밀리초 단위(약 15~20ms 간격, 150표본, 단일 세션 1회, 두 컴포넌트

공통)에서는 공백이 없음을 실측으로 확인하였다. 다만 이는 "이 해상도에서 못 찾았다"는 것이지 완전한 증명은 아니다 — 측정 방식 자체가 유저스페이스에서 nc 프로세스를 매번 새로 fork/exec하는 방식이라 그 fork/exec 오버헤드(수 ms)보다 짧은 공백은 원천적으로 관측할 수 없다. 코드상 진짜 위험 구간인 bpf_link__pin/unlink 교체 찰나의 마이크로초 단위 공백 유무는 bpf_ktime_get_ns() 기반의 커널 내부 계측이 별도로 필요하다. (3) 두 컴포넌트 모두 Tetragon의 Restart=on-failure 같은 자동 재시작 기능이 없다 — 이번 수정은 "재시작 전까지 계속 막아준다"는 것이지 "알아서 재시작된다"는 것은 아니다. (4) watched_self[(3.3절)는 kprobe 기반 컴포넌트에서 fork 없는 self-exec 치환 케이스를 탐지하지 못한다는 것이 이번에 새로 확인되었다 — LSM 컴포넌트는 혹 타이밍 덕분에 이 케이스도 잡지만 kprobe 컴포넌트는 별도 보완이 필요하다(→ 3.10절 (6차)에서 해소). (5) 데몬 독립성 확인은 trust-add/del(신뢰 IP)에 한해서만 직접 검증하였다 — parent-add/self-add/bin-add/exempt-add/cgroup-exempt-*도 3.7~3.8절에서 확인한 것과 동일한 핀된 맵 기반 코드 경로를 공유하므로 같은 수준으로 동작할 것으로 예상하나, 각각을 데몬 사망 상태에서 개별 재검증하지는 않았다.

3.10 6차 — watched_self[]의 fork 없는 self-exec 치환 탐지

발견 경위: 3.9절의 재시작 핸드오프 재측정 중, kshield_vpatch(kprobe) 재현에 파이프 없는 단일 명령 (/tmp/raylet -c "nc -w 2 <IP> 80")을 쓰면 공격이 전혀 탐지되지 않는 현상을 우연히 발견하였다. 원인은 bash가 마지막(유일한) 명령을 실행할 때 fork 없이 자기 자신을 그 명령으로 execve() 치환하는 최적화를 수행하기 때문이다 — /tmp/raylet(파일명만 raylet로 바꾼 bash)가 nc로 치환되면서 같은 PID가 그대로 유지된다. tp/sched/sched_process_exec 혹은 execve()가 완료된 이후에 발동하므로, 그 시점엔 이미 comm이 raylet에서 nc로 바뀌어 있어 watched_self[(3.3절) 매칭이 실패하고, fork가 없었으니 ai_worker_lineage에도 등록되지 않아 두 조건 모두 거짓이 된다 — SHADOW_EXEC/SHADOW_CONNECT 어느 쪽도 이 프로세스를 감시 대상으로 인식하지 못한다. 반면 kshield_vpatch_lsm의 bprm_check_security는 execve() 이전에 발동하므로 그 시점엔 comm이 아직 raylet과 같은 watched_self[] 검사가 그대로 성공한다

— 같은 판정 로직이 혹의 실행 시점 차이만으로 kprobe 컴포넌트에서만 무력화되는 사례다.

수정: 커널 BPF 프로그램에 새 혹을 하나 추가한다. tp/syscalls/sys_enter_execve는 execve() 진입 시점 (아직 comm이 바뀌기 전)에 발동하므로, 이 시점의 comm이 watched_self[]와 일치하면 실제 exec가 완료되기도 전에 미리 해당 PID를 ai_worker_lineage에 등록해 둔다. 이후 exec가 실제로 완료되어 trace_shadow_exec/trace_shadow_connect_v4/v6가 발동할 때는 이미 ai_worker_lineage에 등록된 상태이므로, 그쪽 코드는 전혀 손대지 않고도 정상적으로 탐지한다.

VM 실측 결과: 수정 후 재빌드하여 같은 시나리오를 재현하였다.

[대몬 기동] 핀 파일 6개(기존 5개 + kshield_link_lineage_selfexec) 생성 확인

[파이프 없는 단일 명령 self-exec, 수정 전에는 무탐지였던 케이스]

```
$ /tmp/raylet -c "nc -w 2 203.0.113.1 80"
```

Killed

소요시간: 0.006초 (수정 전 2.008초 → 수정 후 즉시 차단)

[회귀 확인 1: 정상 job]

```
$ /tmp/raylet -c "echo benign-job"
```

benign-job (오탐 없이 정상 통과)

[회귀 확인 2: 기존 fork 강제 케이스(3.9절 방식)]

```
$ /tmp/raylet -c "nc -w 2 203.0.113.1 80 | cat >/dev/null"
```

소요시간: 0.007초 (기존과 동일하게 즉시 차단)

[대몬 로그]

```
SHADOW_EXEC 탐지! parent=bash(pid=8279) -> child=nc(pid=17532) exec=/usr/bin/nc => SIGKILL 전송
```

```
SHADOW_EXEC 탐지! parent=raylet(pid=17539) -> child=nc(pid=17540) exec=/usr/bin/nc => SIGKILL 전송
```

첫 번째 로그 줄(parent=bash)이 self-exec 케이스다 — parent_comm은 BPF 코드가 항상 real_parent(이 프로세스를 최초로 fork한 로그인 셸)의 comm을 읽으므로 raylet가 아닌 bash로 나타나며, 이는 로그 표기 방식일 뿐 탐지 로직의 오류가 아니다. 두 번째 줄(parent=raylet)은 기존 fork 강제 케이스로, 이번 수정과 무관하게 원래부터 정상 동작하던 경로다. 두 경로 모두 정확히 한 번씩만 로그가 남아 중복 탐지나 누락이 없음을 확인하였다.

남는 한계: (1) execve()만 다루며, execveat()(예: fexecve())를 통한 실행)를 통한 동일한 self-exec 치환은 별도 혹이 없어 여전히 놓칠 수 있다. (2) 이번 검증도

단일 세션 1회 실행 기반이며, N회 반복 확인은 아직 없다.

3.11 7차 — 실제 Ray 클러스터 검증과 IPv4-mapped IPv6 신뢰 우회

발견 경위: 5장 향후 연구에 "실제 Ray 클러스터 환경 검증"을 미해결로 남겨 두었던 것을 이번에 다루었다. mock 서버 대신 실제 Ray 2.52.0을 VM에 설치하고 로컬 클러스터(ray start --head)를 띄우려 하자, kShield-VirtualPatch(kshield_vpatch)를 켜 둔 상태에서는 클러스터 자체가 기동되지 않았다. 데몬 로그를 확인한 결과, raylet(watched_self[] 기본값에 포함된 이름) 자신이 GCS 서버로 접속하는 시도가 매번 SHADOW_CONNECT로 탐지되어 SIGKILL당하고 있었다 — 원인은 Ray의 GCS 주소가 loopback(127.0.0.1)이 아니라 노드의 실제 네트워크 인터페이스 IP였기 때문이다. trusted_dst_ipv4_map이 기본적으로 비어 있는 상태에서 kShield-VirtualPatch를 재컴파일 없이 그대로 실제 Ray 클러스터에 적용하면, Ray 자신의 정상적인 내부 통신이 공격처럼 보여 클러스터가 아예 뜨지 못한다는 뜻이다. 이는 코드 결함이 아니라 "낮선 목적지는 기본적으로 차단한다"는 설계 원칙이 만들어낸 예상 밖의 부작용이며, 실제 배포 전에는 GCS를 비롯한 노드의 IP를 미리 신뢰 목록에 등록해야 한다는 운영 지침이 필요함을 보여준다.

GCS IP를 kshield_ctl trust-add로 등록하자 raylet은 정상적으로 기동되었으나, 이번에는 Ray가 새로 생성하는 워커 프로세스(python3.11)가 간헐적으로 여전히 SIGKILL당하는 현상이 남았다. 데몬 로그에는 매번 목적지가 0.0.0.0으로 찍혔는데, 신뢰 목록에 0.0.0.0도 이미 등록해 두었음에도 계속 차단되는 모순이 있었다. bpf_printk 기반 임시 커널 디버그 출력(진단 후 제거)으로 직접 확인한 결과, 이 이벤트는 IPv4 혹은 (trace_shadow_connect_v4)이 아니라 IPv6 혹은 (trace_shadow_connect_v6)에서 발생하고 있었고, 실제 목적지는 ::ffff:223.195.109.35(GCS의 실제 IPv4 주소를 감싼 IPv4-mapped IPv6 주소)였다 — Ray의 gRPC 클라이언트가 같은 목적지에 IPv6 소켓으로도 접속을 시도하고 있었던 것이다. 로그에 목적지가 0.0.0.0으로 잘못 찍힌 것은, IPv6 이벤트도 유저스페이스가 IPv4 전용 필드(evt.dst_addr, IPv6 혹은 채워지지 않아 0으로 남음)로 출력하기 때문임을 함께 확인하였다(별도의 로그 표시 결함). trace_shadow_connect_v6는 애초에 IPv6 목적지용 신뢰 목록이 전혀 없어(loopback ::1만 예외) 이 경로가

무조건 차단되었던 것이며, 같은 디버그 출력으로 IPv4 혹은 쪽은 해당 목적지를 trusted=1로 정확히 인식하고 있음도 함께 확인해, IPv4 신뢰 로직 자체는 처음부터 올바르게 동작하고 있었음을 검증하였다.

수정: trace_shadow_connect_v6에서 목적지 주소가 ::ffff:0:0/96(IPv4-mapped IPv6) 대역인지 확인하고, 해당하면 뒤 4바이트에 담긴 실제 IPv4 주소를 꺼내 기존 trusted_dst_ipv4_map으로 그대로 재판정하도록 하였다 — 별도의 IPv6 전용 신뢰 목록을 새로 만들지 않고 이미 검증된 IPv4 신뢰 로직을 재사용하였다. 순수 IPv6 목적지(IPv4-mapped가 아닌 경우)에 대한 신뢰 목록은 여전히 없다.

VM 실행 결과: 수정 후 재빌드하여 실제 Ray 클러스터로 전 과정을 재현하였다. (참고: Python 3.12 환경에서는 Ray 자체의 대시보드 서브프로세스가 원인 불명으로 기동에 실패해, kShield-VirtualPatch와 무관한 환경 이슈로 판단하여 Python 3.11 가상환경으로 우회하였다.)

[GCS IP 신뢰 등록 후 클러스터 기동]

```
$ ray start --head
```

```
Ray runtime started.
```

[정상 job 5회 제출, 실제 Jobs API(/api/jobs/, CVE-2023-48022가 노리는 그 엔드포인트)]

```
entrypoint: python3 -c "print(N)" (N=1~5)
→ 5개 전부 SUCCEEDED
```

[악성 job 제출, 같은 API로 재현]

```
entrypoint: nc -w 2 203.0.113.1 80
→ FAILED, exit code 137(SIGKILL)
```

[데몬 자체 로그]

```
SHADOW_EXEC 탐지! parent=sh(pid=72968) -> child=nc(pid=72970)
exec=/usr/bin/nc => SIGKILL 전송
```

정상 job 5개는 전부 성공(오답 없음)하였고, 실제 Jobs API로 제출한 악성 job은 실제 Ray 워커 프로세스가 그 entrypoint를 실행하는 순간 SHADOW_EXEC으로 정확히 탐지·차단됨을 확인하였다 — mock 서버가 아닌 진짜 Ray 클러스터의 실제 job 실행 경로에 대한 탐지 검증이 이번에 처음으로 이루어졌다.

남는 한계: (1) 순수 IPv6(비-mapped) 목적지용 신뢰 목록은 여전히 없다. (2) 실제 배포 시에는 클러스터의 모든 노드 IP(GCS를 비롯한 각 노드의 내부 통신 주소)를 미리 신뢰 목록에 등록해야 한다는 운영 부담이 새로 확인되었다 — 다중 노드 클러스터에서는 이 목록이 상당히 길어질 수 있으며, 자동화(예: Ray 클러스터

설정에서 노드 IP 목록을 읽어 자동 등록)는 아직 없다. (3) 이번 검증은 단일 노드 클러스터·단일 세션 기준이며, mock 서버 기반 실험(3.3절)에서 우려했던 "actor 기반 실행 등 mock과 다른 job 실행 경로" 가운데 단순 python3 -c/서브프로세스 entryptpoint만 확인하였다 — Ray actor, @ray.remote 클래스, 멀티프로세싱 기반 워크로드는 아직 검증하지 못했다. (4) kshield_vpatch_lsm에 대한 실제 Ray 클러스터 검증은 이어서 3.12절(8차)에서 다룬다.

3.12 8차 — LSM 컴포넌트의 실제 Ray 클러스터 검증과 동일한 IPv4-mapped IPv6 우회

발견 경위: 3.11절(7차)에서 검증한 것은 kprobe/tracepoint 기반 메인 구현(kshield_vpatch) 뿐이었다. 이번에는 이미 기동되어 있던 같은 실제 Ray 클러스터(단일 노드, Python 3.11)에 LSM 혹은 사전 차단 구현(kshield_vpatch_lsm, 3.5절)을 추가로 붙여 검증하였다. 커널이 CONFIG_BPF_LSM=y이고 활성화 LSM 목록에 이미 bpf가 포함되어 있어(사전 확인함), 별도의 부팅 설정 변경 없이 바로 attach할 수 있었다. LSM 프로그램 4개(fork/exit 계보 추적, bprm_check_security, socket_connect)가 모두 정상 attach되었고, 이미 맺어져 있던 raylet ↔ GCS 연결은 (기존 연결이라 재검사 대상이 아니므로) 영향받지 않았다.

문제는 실제 Jobs API로 job을 제출해 Ray가 새 워커 프로세스를 fork하자 나타났다 — 새로 뜬 워커 (python3.11/event_engine, 부모 raylet)가 GCS로 연결을 시도할 때마다 LSM_CONNECT_BLOCK이 즉시 발동해 connect() 자체가 -EPERM으로 실패했다. trusted_dst_ipv4_map에 노드 IP를 이미 등록해 두었음에도(7차에서 검증된 것과 동일한 절차) 계속 차단되었으며, 로그에는 목적지가 0.0.0.0:6379로 찍혔다 — 7차에서 확인한 것과 정확히 같은 신호였다. 즉 kshield_lsm_socket_connect의 AF_INET6 분기에는 7차에서 고친 IPv4-mapped IPv6 판정 로직이 애초에 없었고(별도의 독립된 BPF 오브젝트라 코드를 공유하지 않으므로), 같은 근본 원인(Ray의 gRPC 클라이언트가 신뢰된 목적지로도 ::ffff:a.b.c.d 형태로 접속을 시도함)이 서로 다른 파일에 반복 작성된 동일한 코드 패턴 때문에 독립적으로 재현된 것이었다. 7차와의 차이는, 이번엔 raylet 자기 자신이 아니라 매번 새로 fork되는 워커들이 걸렸다는 점과, SIGKILL이 아니라 매 시도가 즉시 -EPERM으로 막혀 제출한 job 6개(정상 5 + 악성 1)가 전부 PENDING에서 진행되지 못했다는 점이다.

gcs_server/raylet/dashboard_agent는 기존 연결이 끊기지 않았으므로 클러스터 자체는 죽지 않았다.

수정: kshield_lsm_socket_connect의 AF_INET6 분기에 trace_shadow_connect_v6(7차)와 동일한 로직 — IPv4-mapped 대역(::ffff:0:0/96) 판별 후 뒤 4바이트의 실제 IPv4 주소를 기존 trusted_dst_ipv4_map으로 재판정 — 을 그대로 이식하였다. 별도의 IPv6 전용 신뢰 맵을 새로 만들지 않고 기존 로직을 재사용한 것도 7차와 동일하다.

VM 실행 결과: 재빌드·재기동 후 같은 테스트를 반복하였다.

[수정 전]

```
LSM_CONNECT_BLOCK 탐지! proc=event_engine parent=raylet
dst=0.0.0.0:6379 => connect() -EPERM
(신규 워커가 뜰 때마다 반복 발생 - 정상 job 5개 + 악성 job 1개 전부 PENDING에서 정체)
```

[수정 후 재검증]

```
LSM_CONNECT_BLOCK 로그 전혀 발생하지 않음(GCS 연결 정상 통과)
LSM_EXEC_BLOCK 탐지! proc=sh parent=sh exec=/usr/bin/nc
=> execve() -EPERM
LSM_EXEC_BLOCK 탐지! proc=sh parent=sh exec=/bin/nc =>
execve() -EPERM
```

[실제 Jobs API 최종 결과]

```
정상 job 5개 → 전부 SUCCEEDED
악성 job(entryptpoint: "nc -l -p 9999") → FAILED, exit code 126
driver 로그: "/bin/sh: 1: nc: Operation not permitted"
```

수정 전에는 신규 워커의 GCS 연결이 매번 막혀 job 6개가 전부 PENDING에 정체되었고, 수정 후에는 정상 job 5개가 전부 SUCCEEDED로 끝까지 완료되었으며 악성 job은 execve() 자체가 -EPERM으로 막혀 nc가 단 한 줄도 실행되지 못한 채 exit code 126으로 실패하였다 — 7차의 SHADOW_EXEC(사후 SIGKILL, exit 137)과 달리, 이번엔 사전 차단이라는 LSM 컴포넌트 고유의 성질이 실제 Jobs API 경로에서도 그대로 확인되었다. 클러스터는 테스트 전 과정에서 한 번도 재시작되지 않고 계속 살아있었다.

남는 한계: (1) 3.11절과 동일하게 순수 IPv6(비-mapped) 목적지용 신뢰 목록은 여전히 없다 — 이번 수정도 IPv4-mapped 경로만 해소한다. (2) enforce_mode를 audit-only(0)로 둔 상태에서의 실제 클러스터 동작은 검증하지 않았다. (3) 이번 검증도 3.11절과 동일하게 단순 서브프로세스 entryptpoint(python3 -c, nc)에 국한되며, actor·멀티프로세싱 기반 job 실행 경로는 아직 다루지 않았다.

(4) 동일한 취약 패턴(IPv4-mapped IPv6 미처리)이 서로 다른 두 개의 독립 BPF 오브젝트(kshield_vpatch.bpf.c, kshield_vpatch_lsm.bpf.c)에 각각 존재했다가 순서대로 발견·수정되었다는 사실 자체가, 두 컴포넌트가 유사한 판정 로직을 각자 유지하는 데서 오는 코드 중복·유지보수 위험을 보여준다 — 향후 공통 판정 로직을 공유 헤더/인라인 함수로 통합하는 리팩터링이 필요할 수 있다.

4. 실험

4.1 실험 환경

항목	사양
OS	Ubuntu 22.04 LTS
커널	Linux 6.8.0-138-generic
eBPF 프레임워크	libbpf + libbpf-bootstrap
Mock Ray Jobs API	Python 3 http.server 기반
벤치마크 도구	자체 제작 Python 스크립트, 요청 500회/런

Table 2. 실험 환경

구분	비활성	활성
악성 entypoint(curl sh) 실행 여부	실행됨(차단 수단 없음)	curl exec 시점에 즉시 SIGKILL로 차단
다단계 체인(python3→sh→curl) 탐지	해당 없음	탐지됨 — fork 기반 계보 추적으로 정확히 포착
정상 job(echo, python3 -c ...) 처리	정상 처리	오탐 없이 정상 처리(SHADOW_EXEC 미발생)

Table 3. 공격 탐지 결과

실제 kShield-VirtualPatch 로그:

```
[00:03:55] SHADOW_EXEC 탐지! parent=sh(pid=55945) -> child=curl(pid=55946) exec=/usr/bin/curl => SIGKILL 전송
```

trace 버퍼 분석 결과, 실행 체인은 python3(워커) → fork → sh(계보 편입) → sh exec(통과) → sh → fork → curl(계보 편입, 2단계 추적 성공) → curl exec(계보 소속 + suspicious_bins[] 일치 → SIGKILL)로 진행되었다.

SHADOW_CONNECT(v3) 우회 시나리오 재현 결과: bash의 내장 TCP 리다이렉션 기능(bash -c 'exec 3<>/dev/tcp/1.1.1.1/80; echo leaked >&3')은 curl/nc 같은 별도 바이너리를 실행하지 않으므로 SHADOW_EXEC로는 탐지되지 않는다. SHADOW_CONNECT는 이 시도를 tcp_v4_connect 진입 시점에 정확히 포착하였다.

4.5절의 Falco/Tetragon 비교 실험은 별도 세션(커널 6.8.0-139-generic, Falco 0.45.0)에서 수행하였다 — 정확한 환경 값은 4.5절에 명시한다. 그룹 간 상대 비교는 같은 세션 안에서 이루어지므로 유효하지만, 위 표의 절대 수치와 4.5절의 절대 수치를 서로 비교해서는 안 된다 (한계 1 참고).

4.2 실험 설계

공격 탐지 실험: (1) kShield-VirtualPatch 비활성 상태에서 exploit_shadowray.py로 악성 entypoint(curl ... | sh) 제출 → mock 서버가 이를 실행하는지 확인(기준선). (2) 활성 상태에서 동일 공격 재현 → SIGKILL 발동까지의 시간, 로그 출력 확인. (3) 오탐 측정: 정상 job 처리 중 오작동 여부 확인.

성능 오버헤드 실험: 비활성/활성 상태에서 정상 job 처리 처리량·지연시간을 N회 반복 측정하고 Welch's t-test로 통계적 유의성을 검증하였다.

4.3 공격 탐지 결과

```
[13:36:02] SHADOW_CONNECT 탐지! parent=sh -> proc=bash (pid=118590) dst=1.1.1.1:80 => SIGKILL 전송
```

동일 조건에서 정상 job(echo benign-job)은 어느 쪽도 발생시키지 않고 정상 처리되어, 새 탐지 계층 추가가 기존 정상 경로에 오탐을 유발하지 않음을 확인하였다.

N=30 반복 탐지율 측정: 위 결과는 원래 단일 실행 기반 정성적 확인에 그쳤다. 이를 보완하기 위해 SHADOW_EXEC(nc를 감시 대상 프로세스명(raylet로 위장한 bash)이 self-exec 치환으로 직접 실행)와 SHADOW_CONNECT(curl을 동일하게 self-exec 치환으로 실행해 신뢰 안 된 목적지로 연결 시도)를 각각 30회씩, 그리고 정상 job(echo)을 30회 반복 재현하였다. SHADOW_EXEC 30/30, SHADOW_CONNECT 30/30 전부 탐지·차단되었고, 정상 job도 30/30 오탐 없이 통과하였다 — 실패 사례는 없었다. 소요시간(공격 명령 실행부터 프로세스 종료까지) 분포는

SHADOW_EXEC가 평균 5.01ms(표준편차 0.33ms, 최소 4.63ms~최대 5.88ms), SHADOW_CONNECT가 평균 12.09ms(표준편차 0.73ms, 최소 11.18ms~최대 14.69ms)로, SHADOW_CONNECT가 약 2.4배 더 걸렸다 — SHADOW_EXEC은 `execve()` 직후 트레이스포인트에서 곧바로 판정·SIGKILL이 이루어지는 반면, SHADOW_CONNECT는 `curl`이 URL 파싱·소켓 생성 등을 거쳐 실제로 `connect()`를 호출하는 시점까지 진행된 뒤에야 판정이 이루어지므로, 그 사이 단계가 소요시간에 그대로 더해지기 때문으로 해석된다. 이 재현은 6차(3.10절)에서 고친 self-exec 치환 케이스도 N=30 규모로 재검증한 것을 겸한다.

한계: 위 N=30 측정은 단일 세션·단일 VM 기준이며, `kshield_vpatch_lsm`(3.5절)에 대해서는 아직 동일한 반복 측정을 수행하지 않았다. 또한 여기서 켄 소요시간은 유저스페이스에서 관측한 명령 실행~종료까지의 벽시계 시간이며, SIGKILL 신호 자체가 커널 내부에서 전달되는 마이크로초 단위 정밀 지연시간(`bpf_ktime_get_ns()` 기반)은 아직 별도로 측정하지 않았다. SHADOW_CONNECT의 나머지 한계(기존 연결 재사용, 허용 포트 위 터널링, 네트워크 불필요 공격)는 3.3절에 기술하였다.

4.4 성능 오버헤드 결과

계산 도구 자체의 정확성 검증: 아래 모든 p-value는 `att ack/stat_analysis.py`가 `scipy` 없이 직접 구현한 정규화 불완전 베타함수 기반 t-분포 CDF로 계산된다. 검증 과정에서 `scipy.stats.ttest_ind(equal_var=False)`와 65개 케이스로 대조한 결과, 연속분수 계산이 t값이 0에 매우 가까운 영역에서 수렴하지 않는 오류가 확인되었다(한 케이스에서 `scipy` $p=0.996$ 을 $p=0.640$ 으로 오계산). 표준 대칭 항등식을 적용하여 수정하였고, 65개 케이스 전체에서 `scipy`와 최대 오차 $1e-12$ 이내로 일치함을 확인하였다. 이미 발표되었던 p-value 18개를 버그 수정 전/후로 재계산해 대조한 결과, 실제 측정된 t값들은 버그가 발동하는 극단 영역에 해당하지 않아 기존에 발표한 값들은 영향받지 않았음을 확인하였다.

측정 설계: 같은 세션·같은 시스템 상태에서 세 그룹을 비교하였다 — (1) off: 비활성(baseline), (2) v3: SHADOW_EXEC+CONNECT 실행, (3) v3_lsm: v3에 더해 LSM 혹 기반 사전 차단까지 동시 실행. 각 그룹에서 정상 job을 500회/런 × 10회 반복 제출하여 처리량·지연시간을 측정하였다.

구분	처리량 (req/s)	지연시간 (ms)
off (비활성)	1904.02 ± 6.10	0.484 ± 0.002
v3 (SHADOW_EXEC+CONNECT)	1896.63 ± 11.10	0.486 ± 0.003
v3_lsm (+LSM 사전 차단)	1898.86 ± 8.69	0.486 ± 0.002

Table 4. 비교군별 성능 측정 결과 (N=10, 평균±표준편차)

비교	처리량 t	처리량 p	지연 t	지연 p
off vs v3	+1.845	0.0864 ns	-1.566	0.1400 ns
v3 vs v3_lsm	-0.501	0.6228 ns	+0.083	0.9348 ns
off vs v3_lsm	+1.536	0.1440 ns	-1.784	0.0932 ns

Table 5. Welch's t-test 결과 (ns = 유의하지 않음, $p \geq 0.05$)

세 그룹 간 처리량·지연시간 차이는 어느 쌍을 비교해도 통계적으로 유의미하지 않았다. 단, $p > 0.05$ 는 "차이가 없음"을 증명하는 것이 아니라 "차이를 기각하지 못한 것"임을 유의해야 한다. 완전한 동등성 주장을 위해서는 TOST 기반 동등성 검정과 더 큰 표본($N \geq 30$)이 필요하며, 이는 향후 연구 과제로 남긴다.

한계 1 — 절대 수치의 세션 의존성: Table 4의 절대 처리량은 이전 측정과 크게 다르다. 그룹 간 상대 비교는 유효하지만, 절대 수치 자체는 측정 시점의 시스템 배경 부하에 크게 좌우되는 것으로 보인다.

무관한 프로세스에 대한 영향 측정: `connect` 관련 혹은 AI 워커 계보 여부와 무관하게 시스템의 모든 프로세스의 모든 `connect()` 호출에서 실행된다. 이 "무관한 프로세스" 경로의 오버헤드를 별도로 측정하였다.

구분	처리량 (req/s)	지연시간 (ms)
off_unrelated	2557.70 ± 295.07	0.352 ± 0.051
v3_unrelated	2546.47 ± 206.83	0.350 ± 0.039
v3_lsm_unrelated	2600.01 ± 22.16	0.341 ± 0.003

Table 6. 무관한 프로세스의 `connect()` 성능 (N=10, 평균 ±표준편차)

비교	처리량 t	처리량 p	지연 t	지연 p
off vs v3	+0.099	0.9227 ns	+0.083	0.9347 ns
v3 vs v3_lsm	-0.814	0.4362 ns	+0.773	0.4590 ns
off vs v3_lsm	-0.452	0.6617 ns	+0.693	0.5059 ns

Table 7. Welch's t-test 결과, 무관한 프로세스 (ns = 유의하지 않음, $p \geq 0.05$)

AI 워커 계보와 무관한 connect()에서도 세 그룹 간 유의미한 차이는 관측되지 않았다.

한계 2: 위 측정은 루프백 목적지, 순차 connect() 워크로드만 다룬다. 다수의 무관한 프로세스가 동시다발적으로 connect()를 시도하는 고동시성 상황에서의 영향은 측정하지 않았다.

fork 집약적 워크로드 측정: `--fork-heavy` 옵션으로 job 하나당 `/bin/true`를 50회 반복 fork+exec하는 entripoint를 사용하여 동일한 세 그룹을 재측정하였다.

구분	처리량 (req/s)	지연시간 (ms)
off_fork	605.56 $\pm$ 1.95	1.582 $\pm$ 0.005
v3_fork	603.12 $\pm$ 3.12	1.588 $\pm$ 0.008
v3_lsm_fork	601.51 $\pm$ 1.94	1.593 $\pm$ 0.005

Table 8. fork 집약적 워크로드 성능 측정 결과 (N=10, 평균 $\pm$ 표준편차)

비교	처리량 t	처리량 p	지연 t	지연 p
off vs v3	+2.098	0.0531 ns	-2.235	0.0409 *
v3 vs v3_lsm	+1.382	0.1870 ns	-1.463	0.1644 ns
off vs v3_lsm	+4.648	0.0002 ***	-5.009	0.0001 ***

Table 9. Welch's t-test 결과, fork 집약적 워크로드 (ns = 유의하지 않음, * $p < 0.05$, * $p < 0.001$)**

여기서는 처음으로 통계적으로 유의미한 차이가 관측되었다. off 대비 v3_lsm는 처리량 -0.67%, 지연시간 +0.7%로 그 자체는 작지만, 표준편차가 매우 작아 N=10에서도 강한 유의성($p < 0.001$)으로 검출되었다. v3 단독 대비 v3_lsm 추가분은 유의하지 않아, LSM 계층 자체보다는 v3의 계보 추적 비용이 fork 빈도가 높아지며 누적되어 통계적으로 드러난 것으로 해석된다. **오버헤드가 워크로드의 fork 빈도에 비례하며, fork가 드문 워크로드에서는 노이즈에 묻혀 보이지 않다가 fork가 빈번해지면 드러난다.** 대략적으로 job당 약 50회 늘어난 fork+exit 쌍에 대해 11마이크로초의 추가 지연이 관측되어, 추가 fork+exit 쌍 하나당 약 0.2마이크로초 수준의 비용으로 추산된다.

정밀 per-fork 비용 재검증: 위 0.2마이크로초 추정치를 검증하기 위해, trace_lineage_fork/trace_lineage_exit 두 훅 자체에 bpf_ktime_get_ns() 기반 진입/종료 타임스탬프를 심어 훅 실행 시간만 커널 내부에서 직접 격리 측정하는 hook_timing_map(PERCPU_ARRAY)을 추가하고, 이를 읽는 kshield_ctl fork-timing-stats 명령을 구현하였다. 데몬을 새로 기동한 뒤 위와 동일한 fork 집약적 워크로드(benchmark_vpatch.py --fork-heavy, 500회)를 재현하여 측정한 결과, trace_lineage_fork는 13,528회 호출에 평균 1039.8ns, trace_lineage_exit는 13,528회 호출에 평균 680.6ns가 걸려, fork+exit 한 쌍의 순수 훅 실행 비용은 약 1.72마이크로초로 나타났다 — 위 end-to-end 추정치(약 0.2마이크로초)보다 약 8~9배 크다. 이 차이는 기존 추정 방식이 HTTP 왕복 지연시간이라는 훨씬 거친 단위(밀리초)에서 역산한 값이라, 마이크로초 단위의 실제 훅 비용을 안정적으로 분리해내지 못했음을 보여준다 — 기존 추정치는 방향(오버헤드가 존재하고 fork 빈도에 비례한다는 것)은 맞았지만 절대 크기는 과소평가하고 있었다. 호출 횟수(13,528회)가 이론상 최대치(job당 최대 52회 $\times$ 500 = 26,000회)의 약 절반인 이유는 자원 고갈이 아니라, FORK_HEAVY_ENTRYPPOINTS에 등록된 두 entripoint 중 하나(echo x > /dev/null 반복)가 echo를 외부 바이너리가 아닌 셸 내장 명령으로 실행해 fork를 거의 유발하지 않기 때문이다(job당 약 2회) — 나머지 하나(/bin/true 반복, job당 약 52회)와 50:50으로 섞이며 평균이 약 27회/job으로 낮아진 것이며, 워크로드 구성상 원래 의도된 동작이다.

다중비교 보정과 통계적 검정력에 대한 유의: 본 절까지 Table 5/7/9에서 총 18회의 Welch's t-test를 수행하였다. Bonferroni 보정($\alpha = 0.05/18 \approx 0.0028$)을 적용해 재검토한 결과, off vs v3_lsm은 보정 후에도 유의성이 유지되지만 Table 9의 off vs v3 지연시간($p = 0.0409$)은 보정 기준을 통과하지 못해 유의하지 않은 것으로 재해석해야 한다. 즉 다중비교 보정을 통과하는 유일한 유의미한 차이는 off vs v3_lsm(fork 집약적 워크로드) 하나다. N=10은 검정력이 낮아, "유의하지 않음(ns)" 판정들은 "차이가 없음의 증명"이 아니라 "이 표본 크기로는 작은 차이를 검출하지 못했을 수 있다"는 의미로 읽어야 한다.

4.5 Falco·Tetragon과의 비교 실험

2.3절에서 인정한 공백을 메우기 위해, kShield-VirtualPatch의 SHADOW_CONNECT/SHADOW_EXEC와 같은 판정

(AI 워커 계보에 속한 프로세스가 loopback 밖 IPv4로 connect를 시도하거나 nc/ncat을 exec)을 Falco 룰[5]과 Tetragon 정책[6]으로 각각 옮겨, 같은 VM·같은 세션·같은 워크로드에서 비교하였다. Falco는 modern eBPF 드라이버(0.45.0)로, Tetragon은 standalone 에이전트로 각각 기본 설정으로 실행하였으며, 커스텀 룰/정책 외의 부가 기능은 켜지 않았다. off(도구 없음)를 포함한 네 그룹을 도구를 먼저 기동한 뒤 mock 서버를 띄우는 순서로 구성해, 모든 도구가 계보를 처음부터 동일하게 관찰하도록 통제하였다.

측정 설계상의 특이점: 탐지 실험의 목적지는 애초 외부 IP(1.1.1.1)였으나, 실험 VM이 외부망에 도달하지 못해

응답 없는 목적지로의 블로킹 connect(bash /dev/tcp)가 제한 시간 안에 반환되지 않는 문제를 실측으로 발견하였다. Falco 룰은 connect 반환 시점에 평가되므로 이 조건에서 탐지를 놓쳤으나, 이는 도구의 한계가 아니라 실험 환경의 결함이었다. VM 자신의 비 loopback IPv4 주소에 하네스가 여는 TCP 수신기(Sink)로 목적지를 바꿔 해소하였다 — 세 도구 모두 127.0.0.0/8 밖의 주소를 신뢰되지 않은 목적지로 판정하므로 기준 자체는 바뀌지 않는다. 아울러 Tetragon 이벤트 스트림이 gRPC 소켓 생성 이전에 시작되면 즉시 종료되어, 정책이 실제로는 SIGKILL을 실행하면서도 탐지가 0으로 기록되는 계측 결함도 발견해 수정하였다.

그룹	시나리오	탐지	SIGKILL	Sink 연결 수립	유출 바이트(회당)
off	curl_untrusted	0/10	0/10	10/10	83
off	bash_devtcp	0/10	0/10	10/10	7
off	nc_untrusted	0/10	0/10	10/10	0
kShield	curl_untrusted	10/10	10/10	10/10	0
kShield	bash_devtcp	10/10	10/10	10/10	0
kShield	nc_untrusted	10/10	10/10	0/10	0
Falco	curl_untrusted	10/10	0/10	10/10	83
Falco	bash_devtcp	10/10	0/10	10/10	7
Falco	nc_untrusted	10/10	0/10	10/10	0
Tetragon	curl_untrusted	10/10	10/10	10/10	0
Tetragon	bash_devtcp	10/10	10/10	10/10	0
Tetragon	nc_untrusted	10/10	10/10	10/10	0

Table 10. 탐지·차단 비교 (N=10 반복, 기준 off)

정상 job(benign_echo)은 네 그룹 모두 탐지 0/10·SIGKILL 0/10으로 오탐이 없었다(행 생략). 세 도구가 같은 판정을 내리는 것은 룰/정책을 동등하게 이식했으므로 당연하다 — 이 표에서 갈리는 것은 판정 결과가 아니라 **차단 시점**이다. kShield와 Tetragon은 유출 바이트가 모두 0으로 첫 데이터가 나가기 전에

차단하였다. Falco는 탐지 전용이라 off와 동일하게 데이터가 그대로 나갔다. nc의 Sink 연결 수립이 kShield에서만 0/10인 것은 SHADOW_EXEC이 exec 단계에서 먼저 차단하기 때문이며, Tetragon 정책에는 대응하는 exec 계층이 없어 connect 단계에서 차단한다(연결은 수립되지만 유출은 0).

그룹	지표	평균 ± SD	Δ%	95% CI	Welch p	TOST p	판정
off	처리량(req/s)	753.27 ± 3.63	(기준)	-	-	-	-
Falco	처리량(req/s)	721.53 ± 4.34	-4.21%	[-4.55, -3.87]	0.0000	1.0000	차이 있음
kShield	처리량(req/s)	749.69 ± 3.13	-0.48%	[-0.76, -0.19]	0.0019	0.0000	마진 이내
Tetragon	처리량(req/s)	729.69 ± 4.99	-3.13%	[-3.50, -2.76]	0.0000	1.0000	차이 있음
off	지연시간(ms)	1.248 ± 0.006	(기준)	-	-	-	-
Falco	지연시간(ms)	1.307 ± 0.008	+4.67%	[+4.31, +5.03]	0.0000	1.0000	차이 있음
kShield	지연시간(ms)	1.255 ± 0.005	+0.50%	[+0.22, +0.79]	0.0010	0.0000	마진 이내
Tetragon	지연시간(ms)	1.291 ± 0.009	+3.39%	[+2.99, +3.79]	0.0000	1.0000	차이 있음

Table 11. 성능 오버헤드 — normal 워크로드 (N=20, 기준 off, Bonferroni $\alpha=0.0042$, 동등성 마진 $\pm 2\%$)

그룹	지표	평균 ± SD	Δ%	95% CI	Welch p	TOST p	판정
off	처리량(req/s)	757.38 ± 3.20	(기준)	-	-	-	-
Falco	처리량(req/s)	715.62 ± 2.81	-5.51%	[-5.77, -5.26]	0.0000	1.0000	차이 있음
kShield	처리량(req/s)	755.08 ± 3.73	-0.30%	[-0.60, -0.01]	0.0428	0.0000	동등
Tetragon	처리량(req/s)	699.56 ± 5.00	-7.63%	[-7.99, -7.28]	0.0000	1.0000	차이 있음
off	지연시간(ms)	1.247 ± 0.005	(기준)	-	-	-	-
Falco	지연시간(ms)	1.322 ± 0.005	+6.04%	[+5.77, +6.32]	0.0000	1.0000	차이 있음
kShield	지연시간(ms)	1.250 ± 0.007	+0.29%	[-0.03, +0.60]	0.0719	0.0000	동등
Tetragon	지연시간(ms)	1.351 ± 0.010	+8.39%	[+7.96, +8.81]	0.0000	1.0000	차이 있음

Table 12. 성능 오버헤드 — fork 집약적 워크로드 (N=20, 기준 off)

그룹	워크로드	p99 지연(ms)	CPU s/런	RSS(MB)	실패 요청
off	normal	1.452 ± 0.047	-	-	0
Falco	normal	1.518 ± 0.038	0.148	327	0
kShield	normal	1.455 ± 0.046	0.000	3	0
Tetragon	normal	1.498 ± 0.039	0.483	112	0
off	fork	1.808 ± 0.067	-	-	0
Falco	fork	1.888 ± 0.084	0.522	327	0
kShield	fork	1.823 ± 0.088	0.000	3	0
Tetragon	fork	2.286 ± 0.173	2.715	300	0

Table 13. p99 지연시간 및 도구 사용자 공간 자원(참고)

해석: 동등성이 입증된 것은 kShield-VirtualPatch뿐이다 — normal 워크로드에서는 통계적으로 유의하지만 사전에 고정한 $\pm 2\%$ 마진 안 (TOST 유의), fork 워크로드에서는 유의성 자체가 사라지며 동등성이 입증되었다. Falco와 Tetragon은 두 워크로드 모두 마진을 벗어나는 유의한 오버헤드를 보였다. 애초 "큰 차이가 없다"는 가설로 이 실험을 설계했으나, 실측 결과는 그 가설을 지지하지 않는다 — 특히 Tetragon은 fork 집약적 워크로드에서 처리량 -7.63%까지 벌어지고 CPU s/런이 normal 0.483s에서 fork 2.715s로 5.6배, RSS도 112MB에서 300MB로 늘었다. kShield의 CPU s/런이 두 워크로드 모두 0.000인 것은 측정 결함이 아니라 설계의 결과에 가깝다 — BPF 프로그램 자체의 실행 시간은 트리거한 프로세스

(mock 서버)에 계상되고, 데몬은 perf buffer 이벤트만 읽는 최소 구현이기 때문이다.

이 차이를 "Tetragon이 느리다"로 단순화해서는 안 된다. Tetragon은 정책과 무관하게 프로세스 생명주기 전체를 관찰하는 범용 에이전트이며, Falco도 사용자 공간에서 이벤트를 필터링하는 범용 구조다. 두 도구 모두 이번 정책/룰 하나만이 아니라 훨씬 넓은 기능 범위를 상시 유지하는 비용을 함께 지고 있을 가능성이 높다(아래 대조군 실험으로 검증). kShield가 가벼운 것은 "더 빠른 구현"이어서가 아니라 AI 워커 게보라는 좁은 판정 하나만 커널 안에서 수행하도록 특화되어 있기 때문이며, 이는 범용성을 포기한 대가로 읽어야 한다.

그룹	워크로드	디스크 write (KB/런)
Falco	normal	0.0
kShield	normal	0.0
Tetragon	normal	1,491.0
Falco	fork	0.0
kShield	fork	0.0
Tetragon	fork	17,134.0

Table 13-1. 도구 자신의 디스크 write(참고, Table 13-14-15와 다른 별도 세션 — 절대 수치를 직접 비교하지 말 것)

어디에 쓰는지 /proc/<tetragon_pid>/fd를 직접 열어 확인한 결과, /var/log/tetragon/tetragon.log(쓰기 전용 fd)로 밝혀졌다. 이 파일을 대상으로 (1) 아무 요청도 보내지 않은 대기 상태에서 10초 동안 6,735바이트(초당 약 673바이트)가 늘었고, (2) 목업 서버에 정상 job 50개를 보내자 183,116바이트(job당 평균 약 3,662바이트)가 추가로 늘었다. 즉 이 쓰기는 (a) 본 실험과 무관하게 VM 안의 다른 프로세스만으로도 상시 발생하고, (b) 우리 job이 만드는 fork/exec/exit 이벤트 수에 비례해서도 늘어난다 — tetra getevents로 누가 실제 구독하고 있는지와 무관하게(이 측정에서는 구독 클라이언트를 띄우지 않았다) 에이전트가 기본으로 커두는 로컬 export 로그로 보인다.

4.5.1 오버헤드 원인 분리: 룰/정책의 비용인가, 에이전트 자체의 비용인가

디스크 I/O로 원인 하나 더 확인: 위 CPU·RSS만으로는 세 도구가 정확히 어떤 자원을 쓰는지 전부 드러나지 않는다. 같은 판정 로직을 별도 세션에서 재측정하면서 /proc/{pid}/io(런 전후 read_bytes/write_bytes 델타)를 추가로 켜 결과(Table 13-1), kShield와 Falco는 디스크 write가 사실상 0이었던 반면 Tetragon만 normal 워크로드(요청 500회/런)에서 런당 약 1,491KB, fork 집약적 워크로드에서 약 17,134KB를 디스크에 썼다 — fork로 job당 프로세스 수가 늘어난 비율과 비슷하게 쓰기 양도 함께 늘었다.

위 해석은 "두 도구 모두 넓은 기능 범위를 상시 유지하는 비용을 지고 있을 가능성이 높다"는 추정에 기대고 있었다. 검증하지 않으면 4.5절의 결과는 "AI 서버 전용으로 좁게 쓴 kShield의 룰이 애초에 더 가볍게 설계되었을 뿐"이라는 반박에 열려 있다 — 그렇다면 본 논문이 한 일이 "eBPF로 가상 패치를 구현"한 것이 아니라 "AI 서버 전용 탐지 룰을 하나 작성"한 것으로 축소된다. 이를 검증하기 위해 룰/정책을 제거한 대조군을 추가했다. falco_norules는 Falco를 결코 매칭되지 않는 더미 룰 하나(proc.pid = -1)로, tetragon_norules는 Tetragon을 TracingPolicy 없이(에이전트만) 기동한다 — 두 경우 모두 이벤트 스트림 자체는 falco/tetragon 그룹과 동일하게 켜져 있다. 나머지 측정 설계(off 기준, ABBA 교차, N=20, Bonferroni·TOST)는 4.5절 본 실험과 동일하다.

그룹	처리량(req/s)	Δ%	Welch p	TOST p	판정	CPU s/런	RSS(MB)
off	745.96 ± 5.11	(기준)	-	-	-	-	-
Falco	718.80 ± 3.69	-3.64%	0.0000	1.0000	차이 있음	0.149	327
Falco(무룰)	700.93 ± 5.81	-6.04%	0.0000	1.0000	차이 있음	0.636	328
Tetragon	726.70 ± 5.53	-2.58%	0.0000	0.9931	차이 있음	0.513	109
Tetragon(무정책)	729.28 ± 5.02	-2.24%	0.0000	0.8610	차이 있음	0.479	94

Table 14. 오버헤드 원인 분리 — normal 워크로드 (N=20, 기준 off, Bonferroni α=0.0031, 동등성 마진 ±2%)

그룹	처리량(req/s)	Δ%	Welch p	TOST p	판정	CPU s/런	RSS(MB)
off	751.40 ± 3.27	(기준)	-	-	-	-	-
Falco	708.66 ± 3.14	-5.69%	0.0000	1.0000	차이 있음	0.533	327
Falco(무룰)	682.28 ± 6.29	-9.20%	0.0000	1.0000	차이 있음	0.890	331
Tetragon	692.97 ± 6.28	-7.78%	0.0000	1.0000	차이 있음	2.751	299
Tetragon(무정책)	693.35 ± 5.38	-7.73%	0.0000	1.0000	차이 있음	2.732	292

Table 15. 오버헤드 원인 분리 — fork 집약적 워크로드 (N=20, 기준 off)

해석: 두 도구가 반대 양상을 보였다. **Tetragon**은 정책 유무와 무관하게 처리량·CPU·RSS가 거의 동일했다 (normal 처리량 -2.58%→-2.24%, CPU 0.513s→0.479s, RSS 109MB→94MB; fork 처리량 -7.78%→-7.73%, CPU 2.751s→2.732s, RSS 299MB→292MB) — 오버헤드가 정책 내용이 아니라 에이전트가 정책과 무관하게 상시 수행하는 프로세스 생명주기 관측 자체에서 온다는 가설을 뒷받침한다. **Falco**는 반대였다 — 룰을 제거하자 오버헤드가 줄지 않고 오히려 커졌다(normal CPU 0.149s→0.636s, 4.3배; 처리량 -3.64%→-6.04%; fork CPU 0.533s→0.890s, 1.7배; 처리량 -5.69%→-9.20%). RSS는 룰 유무와 거의 무관했다(327MB ↔ 328MB, 327MB ↔ 331MB) — 에이전트가 유지하는 프로세스/컨테이너 상태 자체는 룰 내용과 무관하다는 뜻이다.

이 "정책과 무관한 상시 비용" 해석은 위 Table 13-1의 디스크 I/O 추가 측정으로도 뒷받침된다. 이번 perf 측정에는 TracingPolicy가 매치될 공격 시나리오 자체가 없었는데도 Tetragon은 계속 디스크에 썼고, 실제로 /var/log/tetragon/tetragon.log라는 구체적인 파일에 쓰고 있음을 직접 확인하였다 — 정책 내용과 무관한 비용이 CPU·RSS뿐 아니라 디스크 쓰기라는 물리적 형태로도 나타난다는 뜻이다. 다만 이 export 로그를 끄거나 로테이션을 설정했을 때 오버헤드가 줄어드는지는 검증하지 않았고(설정 가능 여부 포함 향후 과제), 측정 시점 기준 이 파일이 로테이션 없이 8.4MB 넘게

누적되어 있어 운영 시 디스크 사용량 관리도 함께 고려해야 함을 시사한다.

Falco 쪽 결과는 애초 세운 가설("룰을 걷어내면 가벼워질 것")과 반대였다. 정확한 원인은 이 실험 설계로는 확정할 수 없으나, 하나의 가능한 설명(추정, 미검증)은 이식한 실제 룰이 evt.type = connect / evt.type in (execve, execveat)로 이벤트 유형을 먼저 좁히는 반면, 이벤트 유형과 무관하게 항상 평가되는 더미 룰(proc.pid = -1)은 더 넓은 범위의 이벤트에 대해 룰 평가 자체가 수행되도록 만들었을 가능성이다 — 사실이라면 "룰이 좁을수록 가볍다"가 아니라 "룰이 이벤트 유형으로 얼마나 먼저 걸러지는가"가 관건이라는 뜻이 된다.

두 결과를 합치면 4.5절 서두의 추정을 반박하는 결론이 나온다:

kShield-VirtualPatch가 Falco/Tetragon보다 100배 가까이 가벼운 것은, AI 서버 전용으로 좁게 쓴 룰 덕분이 아니다. Tetragon은 정책 내용과 무관하게 무겁고, Falco는 룰을 없애도 가벼워지지 않는다(오히려 무거워졌다) — 두 결과 모두 "룰을 좁게 썼기 때문에 가볍다"는 가설과 맞지 않는다. 이 격차는 세 도구의 **아키텍처 차이**에서 온다. Falco와 Tetragon도 kShield와 마찬가지로 eBPF 기반이므로 (Falco는 modern eBPF 드라이버, Tetragon은 애초에 eBPF 전용으로 설계됨) 이 격차는 "eBPF 대 비eBPF"의 차이가 아니다. 차이는 두 도구가 지금 로드된 룰/정책이 무엇이든 상관없이 앞으로 올 수 있는 임의의 룰을 지원하기 위한 범용 관측 인프라(프로세스/컨테이너 상태

추적, 이벤트 스트림·gRPC 서버, 룰 엔진)를 상시 유지해야 하는 반면, kShield는 AI 워커 계보에 대한 고정된 판정 하나만 커널 안에서 내리고 매칭된 이벤트만 사용자 공간으로 올리는 최소 데몬이라는 데서 온다(3.2~3.3절). 즉 이 실험이 분리해 낸 것은 **범용 확장형 관측 플랫폼과 단일 고정 판정 간의 구조적 비용 차이**이며, 2.3절·본 절 서두에서 제시한 "범용성의 비용 대 특화의 이점" 해석을 실측으로 뒷받침한다.

범위와 한계: (1) 감시 루트는 python3이며 IPv4 한정이다. (2) Tetragon 정책에는 exec 계층이 없어 nc도 connect 단계에서 차단되므로, Table 10의 "연결 수립" 차이는 도구의 우열이 아니라 계층 설계의 차이이다. (3) kShield의 신뢰 IP·예외 UID·cgroup 예외·syslog 연동 같은 운영 기능(3.9~3.10절)은 이 실험이 비교하지 않는다. (4) 위 결과는 VM 1대·세션 1회의 측정이다. 4.4절의 절대 수치와 마찬가지로 세션 간 절대 비교는 하지 않았으며, 여러 세션에 걸친 반복 측정은 향후 연구로 남긴다(5장). (5) 4.5.1절의 Falco "이벤트 유형 필터링" 설명은 libsinsp 내부 계층으로 확인하지 않은 추정이다 — 코드 수준 검증은 향후 연구로 남긴다. (6) Table 13-1의 디스크 I/O는 Table 13·14·15와 다른 별도 세션에서 추가로 측정한 것이라 CPU·RSS 절대 수치가 정확히 일치하지 않는다(이 세션의 Tetragon RSS는 normal 118MB·fork 470MB로, Table 13의 112MB·300MB와 다르다) — 상대적 경향(정책 무관, Tetragon만 유의미한 디스크 비용)만 Table 13·14·15와 일관되게 재확인된 것으로 읽어야 한다.

4.6 다른 CVE·프레임워크로의 일반화 검증 (예비 실험)

3.1절은 "ShadowRay 외 다른 CVE에 대한 일반화된 방어는 본 논문의 범위 밖"이라고 명시하였다. SHADOW_EXEC/SHADOW_CONNECT의 판정 로직 자체는 "어떻게 코드 실행 권한을 얻었는가"가 아니라 "그 권한으로 무엇을 하려 하는가"만 보므로 원리적으로는 다른 취약점에도 적용될 수 있어야 하지만, 지금까지는 실측으로 확인한 적이 없었다. 이 절은 그 공백을 메우기 위해, 커널 코드(kshield_vpatch.bpf.c)를 한 줄도 수정하지 않고 kshield_ctl parent-add(3.7~3.8절의 런타임 제어 계층)만으로 다른 프레임워크의 다른 CVE도 막히는지 확인한 예비 실험이다.

대상 선정: ShadowRay와의 구조적 유사성과 노출 조건이 서로 다른 일곱 CVE를 의도적으로 골랐다.

- **CVE-2023-43654(ShellTorch, TorchServe)** [12]: 인증 없는 관리 API가 원격 URL의 모델 아카이브를 등록·로드하면서 즉시 코드가 실행된다 —

ShadowRay와 공격 구조(인증 없는 API → 즉시 코드 실행)는 같지만, 실제로는 Java 프런트엔드가 Python 백엔드 워커를 fork해 그 안에서 핸들러 코드를 실행하는 서로 다른 언어 런타임 간 경계를 가진다. "언어·런타임이 달라도 계보 추적이 끊기지 않는가"를 보기 위한 선택이다.

- **CVE-2024-37054(MLflow)** [13]: mlflow.pyfunc.load_model()이 모델 아티팩트(pickle)를 검증 없이 역직렬화하며 코드가 실행된다. ShadowRay·ShellTorch와 달리 공격이 **요청 한 번으로 끝나지 않는다** — ① 악성 아티팩트를 올려두는 시점에는 아무 행위도 일어나지 않고, ② 그 뒤 누군가 모델을 로드(역직렬화)하는 순간에만 코드가 실행된다. "위험해 보이는 데이터가 존재하는 것"이 아니라 "실제 행위가 발생하는 순간"만 판정 기준으로 삼는 kShield·VirtualPatch의 설계 철학(2.4절)이 이런 지연 실행형 취약점에서도 성립하는지 보기 위한 선택이다.
- **CVE-2023-31036(Triton Inference Server)** [14]: 모델 저장소 로드 API(POST /v2/repository/models/<name>/load)의 상대 경로 순회로, 저장소 밖의 임의 경로를 모델 디렉터리로 지정해 그 안의 Python 백엔드 코드를 실행시킨다. 앞의 두 사례와 달리 **기본 상태에서는 발동하지 않고, 운영자가 --model-control explicit(모델을 재시작 없이 올리고 내리기 위한 비기본 옵션)를 켜고 있을 때만** 노출된다 — "기본으로 열려 있는 API"(ShadowRay·ShellTorch)나 "저장소 쓰기 권한만 있으면 되는 경우"(MLflow)와는 또 다른 노출 조건에서도 탐지가 성립하는지 보기 위한 선택이다.
- **CVE-2025-66448(vLLM)** [15]: trust_remote_code=False로 설정해 신뢰 안 된 모델 코드 실행을 막았다고 믿는 상태에서도, 모델의 auto_map 필드가 별도의 "백엔드" 저장소를 가리키면 get_class_from_dynamic_module이 그 저장소의 코드를 검증 없이 실행한다. 앞의 세 사례와 달리 **운영자가 이미 안전 조치를 취했다고 믿는 상태에서 우회되는 취약점**이다 — 3.1절에서 다른 CVE-2025-62593(2.52.0 이전 구버전에만 남아 있던 약한 보호 장치가 실제로 공격받아 CISA KEV에 오른 사례)과 마찬가지로, "완화책이 있다고 해서 방심할 수 없고 행위 기반 탐지가 계속 필요하다"는 주장을 다른 프레임워크에서도 검증하기 위한 선택이다.

- **CVE-2024-2912/CVE-2025-27520(BentoML)** [16]: Content-Type:

CVE-2024-1561(Gradio) [17]:
/component_server 엔드포인트가 Component 클래스의 어떤 메서드든 공격자가 지정한 인자로 호출하도록 열려 있다. 앞의 여섯 사례가 전부 "모델을 등록·로드하면 그 안의 코드가 실행된다"는 형태였던 반면, 이건 **모델 로딩과 무관하게 서버 자신이 노출한 RPC 스타일 엔드포인트가 임의 메서드 호출을 허용하는 다른 유형의 취약점**이다 — 공격 벡터의 "형태" 자체가 달라져도 탐지가 성립하는지 보기 위한 선택이다.

CVE-2025-12487/CVE-2025-12488(oobabooga text-generation-webui) [18]: 모델 로드 API가 trust_remote_code 값을 요청 파라미터로 그대로 받아 쓴다. vLLM(CVE-2025-66448)이 "이미 꺼진 안전장치를 우회"하는 경우였다면, 이건 **안전장치를 공격자가 요청에서 그냥 켤 수 있는 더 근본적인 경우**다 — "우회"와 "노출" 두 가지 실패 양상

실제 프레임워크들을 설치하는 대신 3.4절과 동일한 원칙으로

목업	서버
attack/mock_torchserve_server.py,	
attack/mock_mlflow_server.py,	
attack/mock_triton_server.py,	
attack/mock_vllm_server.py,	
attack/mock_bentoml_server.py,	
attack/mock_gradio_server.py,	
attack/mock_textgenwebui_server.py)	를 만들어 각

취약점의 핵심 동작(공격자 제어 API가 공격자 지정 명령을 실행시킨다)만 재현하였다. Triton의 경로 순회, vLLM의

해석
auto_map/get_class_from_dynamic_module

로직, Gradio의 실제 Component 메서드 디스패치처럼 정확한 내부 메커니즘 자체는 재현하지 않고, "그 결과로 공격자 제어 코드가 실행된다"는 결과만 재현하였다. 일곱

목업 서버는 감시 대상 프로세스명 자체가 기본값
(watched_parents[])에 없는 "torchserve", "mlflow", "tritonserver", "vllm", "bentoml", "gradio", "textgenwebui"라는 이름으로 실행되며, kshield_ctl parent-add로 런타임에 등록해야만 탐지 대상이 된다 — 등록하지 않으면 탐지되지 않는 것이 정상이다.

CVE	프레임워크	공격 형태	계보	결과
CVE-2023-43654	TorchServe	즉시 실행	torchserve → python3(위커) → sh → nc (3단계)	SHADOW_EXEC 탐지·SIGKILL
CVE-2024-37054	MLflow	지연 실행(업로드 후 로드 시)	mlflow → sh → nc (2단계)	업로드: 탐지 없음(정상) → 로드: SHADOW_EXEC 탐지·SIGKILL
CVE-2023-31036	Triton	즉시 실행(비기본 옵션 하에서)	tritonserver → python3(백엔드 스텝) → sh → nc (3단계)	SHADOW_EXEC 탐지·SIGKILL
CVE-2025-66448	vLLM	즉시 실행(안전 플래그 우회)	vllm → sh → nc (2단계)	SHADOW_EXEC 탐지·SIGKILL
CVE-2024-2912/2025-27520	BentoML	즉시 실행 (역직렬화, 단일 요청)	bentoml → sh → nc (2단계)	SHADOW_EXEC 탐지·SIGKILL
CVE-2024-1561	Gradio	즉시 실행(임의 메서드 호출)	gradio → sh → nc (2단계)	SHADOW_EXEC 탐지·SIGKILL
CVE-2025-12487/12488	text-generation-webui	즉시 실행(안전 플래그 노출)	textgenwebui → sh → nc (2단계)	SHADOW_EXEC 탐지·SIGKILL

Table 16. 다른 CVE·프레임워크 일반화 예비 검증 결과

TorchServe·Triton의 3단계 계보 (torchserve/tritonserver → python3 워크스텝 → sh → nc)는 ShadowRay의 2단계 계보(raylet → sh → curl)보다 한 단계 더 깊은데도 ai_worker_lineage 추적이 끊기지 않았다. MLflow는 악성 아티팩트를 올린 직후 journalctl 조회에서 이벤트가 전혀 없었고(대조군), 로드 요청을 보낸 시점에만 탐지되었다 — 데이터가 존재하는 것과 그 데이터로 인해 실제 행위가 발생하는 것을 정확히 구분함을 확인하였다. vLLM·text-generation-webui는 애플리케이션 계층의 안전장치(trust_remote_code)가 우회되었든(vLLM) 애초에 노출되었든(text-generation-webui) 똑같이 탐지되었다 — SHADOW_EXEC/SHADOW_CONNECT는 그 안전장치가 실제로 작동했는지와 무관하게, 안전장치를 통과한 뒤의 행위만 보고 판단하기 때문이다. Gradio는 취약점의 형태 자체가 "모델 로딩"이 아닌 "임의 메서드 호출"이었는데도 결과가 동일한 exec/connect 행위로

귀결되는 한 똑같이 잡혔다 — 탐지가 공격 벡터의 종류가 아니라 그 결과 행위만 본다는 설계 의도(3.3절)를 다시 확인해 준다.

4.6.1 Falco·Tetragon과의 탐지 비교

이어서 같은 일곱 사례에 대해 Falco 를 (kcmp_ai_worker_names 목록에 일곱 이름 추가)과 Tetragon 정책(matchBinaries에 일곱 경로 추가)도 동일하게 확장해, kShield-VirtualPatch를 정지한 독립된 세션에서 두 도구가 각각 탐지·차단하는지 확인하였다(4.5절의 "한 번에 한 도구만 켜다" 원칙과 동일). Table 16과 달리 이번에는 신뢰 안 된 목적지로의 실제 connect까지 발생시키는 명령(nc -w 2 203.0.113.1 80 — 203.0.113.0/24는 RFC 5737 문서화 전용 예약 대역)을 썼다. SHADOW_CONNECT/Tetragon의 connect 계층까지 함께 확인하기 위해서다.

프레임워크	Falco	Tetragon
TorchServe	KCMP_SHADOW_EXEC·CONNECT 탐지	connect 탐지 → SIGKILL
MLflow	KCMP_SHADOW_EXEC·CONNECT 탐지	connect 탐지 → SIGKILL
Triton	KCMP_SHADOW_EXEC·CONNECT 탐지	connect 탐지 → SIGKILL
vLLM	KCMP_SHADOW_EXEC·CONNECT 탐지	connect 탐지 → SIGKILL
BentoML	KCMP_SHADOW_EXEC·CONNECT 탐지	connect 탐지 → SIGKILL
Gradio	KCMP_SHADOW_EXEC·CONNECT 탐지	connect 탐지 → SIGKILL
text-generation-webui	KCMP_SHADOW_EXEC·CONNECT 탐지	connect 탐지 → SIGKILL

Table 17. Falco·Tetragon 탐지 비교 (N=1, connect 포함 시나리오)

일곱 사례 모두 Falco·Tetragon 둘 다 탐지했다 — 두 도구도 "새 프레임워크 이름을 목록에 추가"하는 것만으로 일반화된다는 뜻이다. 다만 그 추가 방식에는 차이가 있었다. kShield-VirtualPatch(kshield_ctl parent-add)와 Tetragon(tetra tracingpolicy add)은 **에이전트를 재시작하지 않고** 새 정책을 반영했다. Falco는 이번 실험이 쓴 기동 방식(롤 파일을 인자로 받아 기동)에서는 **프로세스 자체를 재시작**해야 새 롤이 반영되었다 — Falco가 롤 갱신을 아예 못 하는 것은 아니지만(운영 배포에서는 재적재 메커니즘이 있다), 이 실험 설정에서는 재시작이 필요했다는 점만 사실로 기록한다.

이 재시작 동안 실제로 얼마나 무방비인지를 별도로 측정하였다(N=1, 단일 측정). Falco를 kill -9로 죽이고 곧바로 재기동시키는 순간부터 동일한 공격(nc -w 2 203.0.113.1 80)을 약 0.3초 간격으로 반복 투입하며 탐지

로그를 확인한 결과, 재시작 트리거로부터 3.009초 시점까지는 여전히 탐지되지 않았고 3.465초 시점에는 이미 탐지가 재개되어 있었다 — Falco 자체 로그에 찍힌 커널 이벤트 타임스탬프(evt.time)로 봐도 최초로 잡힌 이벤트는 재시작 트리거로부터 약 3.147초 뒤의 것이었다. 외부에서 폴링한 두 시점과 Falco 자체 타임스탬프가 모두 3.0~3.5초 사이로 일치하므로, 이 실험 설정 기준으로 Falco는 롤 하나를 반영하기 위한 재시작마다 **약 3초의 완전 무방비 구간**이 생기며, 이번 측정에서는 그 사이에 들어온 공격 시도 7회가 전부 놓쳤다. kShield-VirtualPatch·Tetragon은 롤/정책 반영에 재시작 자체가 없으므로 이 구간이 존재하지 않는다. 다만 이는 VM 1대·세션 1회의 단일 측정이며, Falco 재시작 시간은 롤 파일 크기·시스템 부하에 따라 달라질 수 있어 반복 측정이 필요하다.

이 라운드와 Table 16의 차이(정직하게 밝힘): Table 16은 kShield-VirtualPatch 혼자, nc -h(exec만 발생, connect 없음)로 확인한 결과다. 이번 라운드는 kShield-VirtualPatch를 정지한 채 Falco·Tetragon만, connect까지 발생하는 다른 명령으로 확인했다 — **세 도구를 동일한 한 번의 실행으로 나란히 비교한 것이 아니다.** kShield-VirtualPatch 자신의 SHADOW_CONNECT가 이 일곱 프레임워크에서도 작동하는지는 이번 라운드에서 재확인하지 않았다 — 같은 커널 코드가 원본 ShadowRay 실험(4.3·4.5절)에서 SHADOW_CONNECT까지 이미 검증되었고 계보 판정이 SHADOW_EXEC와 동일한 ai_worker_lineage 맵을 공유하므로 작동할 것으로 추정되나, 실측으로 확인한 것은 아니다.

한계 (반드시 함께 읽을 것): (1) Table 16의 각 시나리오에는 **1회씩만** 실행한 기능적 확인이며, 4.6.1절의 Falco·Tetragon 비교도 마찬가지로 1회씩이다 — 4.3절(N=10)·4.4절(N=10)·4.5절(N=20) 같은 반복 통계 측정은 어느 쪽도 하지 않았다. (2) 4.6.1절에서 Falco·Tetragon의 탐지 여부는 확인했지만, 오버헤드·성능 비교(4.5절 수준)는 하지 않았다. (3) 일곱 취약점 모두 실제 프레임워크를 설치하지 않고 묵업으로 핵심 동작만 재현하였다(3.1절의 Ray 토큰 인증 검증만 예외적으로 실제 설치본을 썼다). (4) Table 16의 일곱 사례는 의심 바이너리 실행(SHADOW_EXEC, nc)으로만 구성했다 — SHADOW_CONNECT 경로는 4.6.1절에서 Falco·Tetragon에 대해서만 별도로 확인했고, kShield-VirtualPatch 자신에 대해서는 이번 예비 실험에서 재확인하지 않았다(위 설명 참고). exec/connect 어느 쪽으로도 안 드러나는 공격 형태(3.3절 한계에 이미 명시한 기존 연결 재사용, 로컬 전용 공격 등)는 포함되지 않았다. (5) Gradio의 경우 공개된 실제 공격은 파일 읽기이며, 코드 실행으로 이어지는 구체적 메시드는 공개돼 있지 않다 — "임의 메시지 호출이 가능하다"는 원시적 능력만 실측하고, 그 능력이 셸 명령 실행으로 이어지는 구간은 payload_cmd로 직접 단순화하였다. 따라서 이 절의 결론은 **"같은 커널 로직(kShield)과 같은 방식으로 확장한 룰/정책(Falco·Tetragon)이 형태·노출 조건이 서로 다른 최소 일곱 개의 CVE에도 이름·경로 등록만으로 작동함을 확인"**까지이며, "임의의 CVE를 다 막는다"는 주장이나 세 도구 간 성능·정밀 비교로 확장해서는 안 된다.

4.7 동기 차단과 비동기 SIGKILL의 실제 시간차 실측

3.5절은 bpf_send_signal이 비동기 전달이므로 신호가 실제로 전달되기 전에 execve()/connect()가 이미

진행되었을 가능성을 "이론적으로 완전히 배제하지 못한다"고만 서술하였고, 4.4절은 두 컴포넌트의 처리량·지연시간(정상 트래픽 기준 오버헤드)만 비교하였을 뿐 공격이 실제로 막히기까지 걸리는 시간 자체는 재지 않았다.

4.3절도 SHADOW_EXEC/SHADOW_CONNECT의 소요시간을 잴지만 유저스페이스에서 관측한 명령 실행~종료까지의 벽시계 시간하였고, "SIGKILL 신호 자체가 커널 내부에서 전달되는 마이크로초 단위 정밀 지연시간은 아직 별도로 계측하지 않았다"고 스스로 한계를 명시해 두었다(4.3절 한계). 이 절은 그 공백을 메워, kprobe/tracepoint 기반 비동기 SIGKILL(v3)과 LSM 혹은 기반 동기 -EPERM이 "공격 syscall 진입 → 실제 차단"까지 몇 마이크로초가 걸리는지 커널 내부에서 직접 측정한다.

측정 방법: kShield-VirtualPatch·Falco·Tetragon 중 어느 쪽 코드도 아닌 독립된 bpftrace 스크립트(attack/compare/latency_probe.bt)로 관측하였다 — 측정 대상 도구의 자체 로그를 신뢰하는 대신, 커널 tracepoint/kprobe만으로 제3자 시점에서 재는 것이 목적이다. sys_enter_execve/sys_exit_execve 구간에서 execve() 자체가 즉시 실패하면(반환값 < 0) 그 구간을 **동기 차단(sync_block)**으로 기록하고, 성공하면 이후 do_exit이 호출되는 시점까지를 추적해 **비동기 사후 종료(post_exit)**로 분류하였다. post_exit은 종료 코드에서 시그널 번호를 추출해 SIGKILL(9)로 실제로 죽었는지, 아니면 그냥 정상 종료(0, 차단 아님)인지 반드시 구분하였다 — 차단하지 않는 도구를 같은 스크립트로 재면 post_exit이 전부 sig=0으로 나와야 정상이며, 이를 "느린 차단"으로 잘못 집계하지 않기 위해서다. 공격 시나리오는 4.5절과 동일한 nc_untrusted(nc -w 2 < 신뢰 안 된 IP> <포트>)를 mock Ray Jobs API로 30회 제출하였다.

개발 중 두 가지 실수를 직접 발견·수정하였다: (1) do_exit(long code)의 인자는 함수 본문 시작 시점해야 task_struct.exit_code에 대입되는데, kprobe는 함수 진입(본문 실행 전)에 걸리므로 그 필드를 읽으면 항상 갱신 전의 옛 값(대개 0)이 나와 SIGKILL로 죽었어도 매번 "안 막힘"으로 잘못 분류되는 문제를 첫 실행에서 발견하였다 — v3 자체 로그는 30/30 모두 SIGKILL 전송을 정확히 기록하고 있었는데도 관측 스크립트만 sig=0으로 찍어, 관측 도구의 버그임을 먼저 확인한 뒤 인자 레지스터(arg0)를 직접 읽도록 고쳤다. (2) 분석 스크립트가 두 그룹을 정확히 같은 이벤트 이름(sync_block vs post_exit)일 때만 비교하도록 짜여

있어, 차단 방식 자체가 다른 두 그룹(LSM은 전부 sync_block, v3는 전부 post_exit)이 애초에 비교되지 않는 문제를 발견하여, "이벤트 이름"이 아니라 "실제로 막혔는가"(sync_block 전부 + post_exit 중 sig=9만) 기준으로 묶도록 고쳤다.

결과 (N=30, execve() 경로만 — 한계 (2) 참고):

그룹	차단 방식	평균 ± SD (μs)	중앙값	범위
kshield_lsm	동기 - EPERM(sync_block)	209.86 ± 31.71	197.71	170.37 ~ 289.88
kshield(v3)	비동기 SIGKILL(post_exit, sig=9 30/30)	359.68 ± 30.15	354.96	278.20 ~ 437.48

Table 18. exec 차단까지 걸린 시간

두 그룹 모두 30번 전부 공격을 놓치지 않았다(v3는 자체 로그의 SIGKILL 전송 30건과 일치, LSM은 sync_block 30건 전부 확인). 차이는 149.82μs(LSM이 약 1.71배 빠름)이며, Welch's t-test로 $p=2.9 \times 10^{-26}$ 에 통계적으로 명백하다. 이 값은 4.3절의 SHADOW_EXEC 평균 5.01ms와 자릿수가 다른데, 서로 다른 것을 켜 값이라 직접 비교해서는 안 된다 — 4.3절은 job 제출부터 프로세스 종료까지의 유저스페이스 벽시계 시간(HTTP 왕복·셸 기동 등 포함)이고, 이 절은 execve()가 커널에서 실제로 성공한 시점부터 do_exit 진입까지만 켜 순수 커널 내부 구간이다.

해석: 절대 시간(수백 마이크로초)은 사람이 체감할 수 있는 크기가 아니며, "LSM이 더 빨라서 실무적으로 이득"이라는 식으로 확대 해석해서는 안 된다. 이 결과의 의미는 속도 차이가 아니라 **성격의 차이**다 — v3는 악성 바이너리(nc)의 execve()가 이미 성공한 뒤에 죽이므로, 이 약 360μs 동안 악성 코드가 실제로 커널 위에서 실행되고 있었다. LSM은 그 execve() 자체를 애초에 실패시키므로, 악성 코드는 단 한 순간도 실행된 적이 없다. 3.5절에서 말로만 서술했던 "비동기 SIGKILL의 잔여 한계"를 여기서 처음으로 정량화하였다.

한계: (1) VM 1대·세션 1회·N=30이며, 4.5절과 같은 방식으로 여러 세션에 걸친 재검증은 하지 않았다. (2) execve() 경로만 측정하였다 — 이번 30회 시행 모두 v3의 SIGKILL이 nc가 connect()를 호출하기 전에 도달해, connect() 단계의 동기/비동기 시간차는 이번 데이터에서 관측되지 않았다(SHADOW_EXEC을 audit-only로 끄고 SHADOW_CONNECT만 켜는 별도 실험이

필요하며, 향후 과제로 남긴다). (3) Falco·Tetragon은 이번 측정에 포함하지 않았다 — 관측 스크립트 자체는 도구에 무관하게 동작하므로 그룹만 추가하면 같은 방법으로 켤 수 있다.

5. 결론 및 향후 연구

본 논문은 AI 서버 프레임워크의 알려진 RCE 취약점(ShadowRay)에 대해, 코드 수정이나 재배포 없이 eBPF로 가상 패치를 적용하는 방법을 제시하였다. SHADOW_EXEC과 SHADOW_CONNECT 두 계층은 반복 횟수가 아닌 단일 이상 행위 발생을 기준으로 하므로, 기존 kShield [8]의 빈도 기반 탐지가 가진 구조적 한계를 보완한다.

VM 실험 결과, python3(워커)→sh→curl로 이어지는 2단계 공격 체인은 curl 실행 시점에(SHADOW_EXEC), bash /dev/tcp/ 내장 기능을 이용한 우회 시도는 연결 시도 시점에(SHADOW_CONNECT) 각각 정확히 차단하였으며, 두 경우 모두 정상 job은 오답 없이 통과함을 확인하였다.

비활성/SHADOW_EXEC+CONNECT/LSM 사전 차단을 포함한 3개 그룹을 10회씩 반복 측정한 결과, fork 단계가 얇은 워크로드와 무관한 프로세스의 connect() 성능 모두에서 통계적으로 유의미한 차이는 관측되지 않았다. 다만 job당 fork를 50배가량 늘린 워크로드에서는 처음으로 통계적으로 유의미한 차이가 나타났다 — 절대 크기는 작지만, 오버헤드가 워크로드의 fork 빈도에 비례함을 보여준다.

4.4절이 정상 트래픽 기준의 처리량·지연시간을 비교했다면, 4.7절은 공격이 실제로 막히기까지 걸리는 시간 자체를 독립된 커널 관측 도구(bpftrace)로 정밀 측정하였다(N=30). LSM의 동기 차단(평균 209.86μs)이 v3의 비동기 SIGKILL(평균 359.68μs)보다 149.82μs 빨랐다(Welch $p=2.9 \times 10^{-26}$). 다만 이 차이의 의미는 속도가 아니라 성격이다 — v3는 execve()가 이미 성공한 뒤 죽이므로 그 사이 악성 코드가 실제로 실행되지만, LSM은 execve() 자체를 실패시켜 애초에 실행되지 않는다. 3.5절에서 서술로만 남겨 두었던 "비동기 SIGKILL의 잔여 한계"를 여기서 처음으로 숫자로 확인하였다.

Falco·Tetragon에 같은 판정을 이식해 같은 VM·세션에서 비교한 결과(4.5절), 사전에 고정된 ±2% 마진 안에서 동등성이 입증된 것은 kShield-VirtualPatch뿐이었다. Falco와 Tetragon은 두 워크로드 모두에서 마진을 벗어나는 유의한 오버헤드를 보였으며, 특히 Tetragon은 fork 집약적 워크로드에서

처리량이 -7.63%까지 벌어졌다. 다만 이는 "범용 도구가 열등하다"가 아니라, 두 도구가 이번 판정 하나보다 훨씬 넓은 기본 관측 범위를 상시 유지하는 데서 오는 비용으로 해석해야 한다 — kShield-VirtualPatch가 가벼운 것은 그 범용성을 포기한 대가다.

개발 과정에서 기본 탐지 구조를 확립하는 과정과, 그 이후 8차례의 설계 개선이 있었다. 기본 구조(3.3~3.5절)는 "직속 부모만 확인"하는 초기 설계가 다단계 공격 체인을 놓치고 정상 job도 오탐하는 문제를 실측으로 발견하여 프로세스 계보(lineage) 추적 방식으로 재설계하는 데서 시작하였다. 이어서 실행 파일 이름 블록리스트가 bash 내장 기능 같은 우회에 취약함을 확인하여, 연결 시도 자체를 감시하는 SHADOW_CONNECT를 추가하였다. 비동기 SIGKILL의 잔여 한계를 해소하기 위해 LSM 혹은 기반 동기적 사전 차단 추가를 구현·검증하는 과정에서는, 셸의 \$PATH 재탐색이 시도하는 동일 파일의 다른 경로 문자열을 놓치는 버그를 발견하였다 — 이는 이미 "검증됨"으로 문서화했던 초기 구현에도 동일하게 존재하던 누락이었다. curl/wget처럼 이중 용도 도구를 실행 파일 이름만으로 즉시 차단하면 정당한 사용까지 오탐 처리되는 문제도 발견하여 판단을 목적지 인지 계층(SHADOW_CONNECT)에 넘기도록 재설계하고, audit-only 배포 방식을 지원하는 enforce_mode 런타임 토글을 추가하였다. 마지막으로 계보 판정 로직이 "감시 대상 프로세스의 자손"만 커버하고 감시 대상 프로세스 자신의 직접 행위는 구조적으로 놓친다는 공백을 발견하여, watched_self[] 라는 별도의 좁은 목록으로 자기 자신의 comm도 확인하도록 보강하였다. 이 경험들은 행위 기반 탐지 메커니즘을 설계할 때 (1) 실행 체인의 깊이, (2) 감시 신호가 "특정 도구의 사용"이 아니라 "행위의 본질적 불변량"에 기반해야 함(따라서 "누구의 자손인가"뿐 아니라 "나 자신이 감시 대상인가"까지 물어야 계보 판정이 완전함), (3) 경로 기반 블록리스트는 모든 문자열 별칭을 빠짐없이 열거해야 함을 반드시 고려해야 함을 보여준다.

이 기본 구조가 자리잡은 뒤로도 설계는 8차례 더 개선되었다(Table 1, 3.6~3.12절). 첫째(1차), 실험이 아닌 기획 관점에서 배포 시나리오를 재검토하는 과정에서 데몬이 클러스터보다 나중에 뜨는 상황을 전혀 고려하지 않았음을 발견하였다(3.6절). 유저스페이스 로더가 기동 시 /proc 을 스캔해 이미 떠 있는 프로세스들의 계보를 직접 채워 넣는 방식으로 해소하였다.

둘째(2차/3차), 탐지 정확도만으로는 실제 운영 조직의 도입을 보장할 수 없다는 실무 도입 관점의 한계를 다루었다(3.7절) — BPF map 기반 런타임 제어 계층, syslog/JSON 연동, UID 단위 감시 예외로 이를 해소하였다.

셋째(4차), 둘째 개선이 신뢰 목적지 IP 하나에만 런타임화를 적용했다는 점을 재검토하여, 나머지 세 배열도 동일한 BPF map 방식으로 전환하고 cgroup 단위 예외를 추가하였다(3.8절). 이는 "실무 도입 장벽"이라는 하나의 한계 항목 안에도 여러 하위 요인이 섞여 있을 수 있어, 일부만 해소한 상태로는 나머지가 재검토 없이 간과되기 쉽다는 점을 보여준다.

넷째(5차), 지금까지의 재검토가 탐지 로직 자체나 그 로직을 둘러싼 운영 인터페이스에 집중되어 있었다면, 이번에는 "탐지 로직이 정확해도 그것을 커널에 붙여 두는 메커니즘 자체가 데몬의 비정상 종료에 강건한가"라는 새로운 축의 질문을 다루었다(3.9절) — Falco-Tetragon과 데몬을 kill -9로 강제 종료했을 때의 동작을 대조하다, kShield-VirtualPatch는 BPF 맵만 편해 두었을 뿐 실제 판정·SIGKILL을 수행하는 프로그램의 부착(link) 자체는 편하지 않아 데몬이 죽는 순간 커널의 탐지 로직까지 통째로 사라지는 fail-open임을 발견하였다(Tetragon은 데몬이 완전히 죽어 있는 상태에서도 커널 쪽 차단이 계속됨을 확인). 다섯 개 프로그램의 link를 전부 bpffs에 편하고, 재시작 시 새 attach를 먼저 만든 뒤에야 옛 핀을 지우는 순서로 보호 공백 없이 교체되도록, 반대로 SIGINT/SIGTERM 같은 의도된 종료에서는 명시적으로 unpin해 실제로 꺼지도록 수정하였다. 수정 후 kill -9에도 5개 프로그램이 전부 잔존하며 공격이 여전히 차단됨을, SIGTERM으로는 핀이 실제로 사라짐을 VM에서 각각 확인하였다. 같은 문제가 SIGKILL이 아닌 동기적 사전 차단(LSM) 컴포넌트(kshield_vpatch_lsm, 3.5절)에도 있는지 이어서 확인한 결과 동일한 fail-open이 재현되었고, 같은 방식으로 고쳐 같은 방법으로 재검증하였다. 이 발견은 탐지 로직의 "정확성"과, 그 정확한 로직을 실제로 커널에 유지시키는 메커니즘의 "강건성"이 완전히 별개의 축이며 따로 검증하지 않으면 놓치기 쉽다는 점을 보여준다 — 그리고 그 강건성 문제는 한 컴포넌트에 국한되지 않고, 같은 아키텍처 패턴(맵만 편하고 attach는 안 편하는)을 공유하는 다른 컴포넌트에도 그대로 옮겨갈 수 있다는 점도 함께 보여준다.

다섯째(6차), 넷째의 재시작 핸드오프 재측정 과정에서, 이번에는 그와 무관한 별개의 탐지 공백을 우연히 발견하였다(3.10절) — kprobe 기반 컴포넌트의

watched_self[] (3.3절) 검사가 fork 없이 자기 자신을 다른 바이너리로 치환하는 self-exec 케이스(bash의 단일 명령 exec 치환 최적화)를 놓친다는 것이다. 원인은 tp/sched/sched_process_exec 혹은 execve() 완료 이후에 발동해 그 시점엔 이미 comm이 바뀌어 있기 때문이며, 같은 검사가 execve() 이전에 발동하는 LSM 컴포넌트에서는 문제없이 통과한다. execve() 진입 시점에 혹은 하나 추가해 comm이 바뀌기 전에 미리 계보에 등록해 두는 방식으로 해소하였고, 기존 exec/connect 판정 코드는 전혀 손대지 않았다. 이 발견은 넷째와 마찬가지로 "같은 판정 로직도 혹은 커널 실행 흐름의 어느 지점에 걸리는가에 따라 결과가 달라질 수 있다"는, 개별 컴포넌트 검증만으로는 드러나지 않는 교차 컴포넌트 비교의 가치를 다시 한번 보여준다 — 두 차례(넷째, 다섯째) 모두 서로 다른 컴포넌트를 나란히 비교하는 과정에서만 드러난 문제였다.

여섯째(7차), 지금까지 mock 서버로만 재현해 오던 검증의 공백 — "실제 Ray 클러스터에서도 이 탐지 로직이 그대로 통하는가" — 을 처음으로 다루었다 (3.11절). 실제 Ray 2.52.0 클러스터를 VM에 띄우려 하자 kShield-VirtualPatch를 켜 상태에서는 클러스터 자체가 기동되지 않는 예상 밖의 결과를 만났다 — watched_self[]의 기본값 "raylet"이 진짜 Ray의 raylet과 정확히 일치했고, raylet이 GCS로 접속하는 트래픽이 loopback이 아닌 실제 네트워크 IP로 나가면서 SHADOW_CONNECT에 의해 즉시 SIGKILL당했기 때문이다. 이는 탐지 로직의 결함이 아니라, "낮선 목적지는 기본적으로 의심한다"는 설계 원칙을 실제 다중 프로세스 분산 시스템에 처음 적용해 보고서야 드러난 배포상의 함정이었다 — GCS IP를 미리 신뢰 목록에 등록해야 한다는 운영 지침이 새로 필요함을 보여준다. 이를 해소한 뒤에도 일부 워커 프로세스가 간헐적으로 죽는 현상이 남아 있었는데, 커널 디버그 출력으로 직접 추적한 결과 Ray의 gRPC 클라이언트가 신뢰된 목적지로도 IPv4-mapped IPv6 주소(::ffff:a.b.c.d)를 통해 접속을 시도하고 있었고, IPv6 경로에는 애초에 신뢰 목록 자체가 없어 무조건 차단되고 있었음을 확인하였다 — IPv4 신뢰 로직 자체는 처음부터 정확히 동작하고 있었다. 뒤에 담긴 실제 IPv4 주소를 꺼내 기존 IPv4 신뢰 목록으로 재판정하도록 고친 뒤, 실제 Jobs API로 제출한 정상 job 5개는 전부 성공하고 악성 job(nc)은 SHADOW_EXEC으로 정확히 차단됨을 확인하였다. 이 발견은 mock 기반 검증이 아무리 촘촘해도, "탐지 대상 프로세스명이 방어 도구 자신이 보호해야 할 정상 인프라의 프로세스명과 우연히 겹칠 수 있다"거나 "실제 분산 시스템은 dual-stack 등 mock이

재현하지 않는 통신 경로를 쓴다"는 것처럼, 진짜 대상 환경에 처음 적용해 보기 전까지는 드러나지 않는 한 축의 위험이 따로 있음을 보여준다.

일곱째(8차), 여섯째에서 검증한 것은 kprobe/tracepoint 기반 메인 구현뿐이었다는 점을 곧바로 이어서 확인하였다(3.12절) — 같은 실제 Ray 클러스터에 LSM 혹은 기반 사전 차단 컴포넌트(kshield_vpatch_lsm)를 붙이자, 서로 다른 독립 BPF 오브젝트임에도 kshield_lsm_socket_connect의 AF_INET6 분기에 여섯째와 완전히 동일한 IPv4-mapped IPv6 신뢰 우회가 각자 따로 존재하고 있었음이 드러났다 — 신규로 fork되는 워커마다 GCS 연결이 -EPERM으로 막혀 제출한 job이 전부 PENDING에 정체되었다. 여섯째와 동일한 판정 로직을 이식하여 해소하였고, 재검증 결과 정상 job 5개는 모두 SUCCEEDED, 악성 job은 execve() 자체가 사전 차단되어 exit code 126으로 실패함을 확인하였다. 이 발견은 넷째(fail-open 패턴)에서와 마찬가지로, 같은 취약 패턴이 한 컴포넌트에서 발견·수정되었다고 해서 유사한 로직을 독자적으로 유지하는 다른 컴포넌트에 자동으로 전파되지 않는다는 점을 다시 한번 보여준다 — 공유되지 않는 중복 로직은 반드시 각각 따로 검증해야 한다.

향후 연구 과제는 다음과 같다. 본 논문의 핵심 주장 자체의 신뢰도에 직결되는 항목과, 적용 범위를 넓히는 지엽적인 항목을 구분하여 제시한다.

핵심 주장의 검증 강화(우선순위 높음)

- **실제 Ray 클러스터 환경 검증의 확장:** 3.11절 (7차)·3.12절(8차)에서 메인 구현과 LSM 컴포넌트 양쪽 모두 실제 Ray 2.52.0 클러스터의 실제 Jobs API를 통해 정상 job 5회 성공·악성 job 1회 차단을 확인하여, 이 항목은 기본적인 수준에서 해소되었다. 다만 (1) 검증한 entrypoint는 두 컴포넌트 모두 단순 서브프로세스 실행(python3 -c ..., nc)뿐이며, Ray actor:@ray.remote 클래스·멀티프로세싱 기반의 더 복잡한 job 실행 경로는 아직 검증하지 못했다 — 3.3절에서 다룬 것과 유사한 계보 추적 사각지대가 이런 경로에 다른 형태로 남아있을 가능성을 배제할 수 없다. (2) 단일 노드 클러스터·단일 세션 기준이며, 다중 노드 클러스터에서의 검증과 그때 필요한 신뢰 목록(각 노드 IP) 등록 부담의 정량화가 필요하다. (3) LSM 컴포넌트의 enforce_mode audit-only 모드는 실제 클러스터에서 검증하지 않았다.

- **오탐 시나리오 확장 검증:** 더 복잡한 합법적 워크로드와의 충돌 가능성은 아직 다양한 시나리오로 검증하지 못했다.
- **kshield_vpatch_lsm의 N회 반복 탐지율 측정과 정밀 SIGKILL 지연시간 측정:** 4.3절에서 kshield_vpatch(SHADOW_EXEC/CONNECT)는 N=30 반복으로 탐지율(30/30, 30/30)과 소요시간 분포를 확인하였다. 마이크로초 단위 정밀 지연시간 계측은 4.7절에서 독립된 bpftrace 관측 도구로 수행하였고(exec 경로, N=30, v3 대 LSM), 그 과정에서 kshield_vpatch_lsm의 exec 차단도 30/30 확인되어 이 컴포넌트에 대한 반복 확인이 부분적으로 이뤄졌다. 다만 4.7절은 nc_untrusted 한 시나리오·exec 경로에 한정되며, 4.3절 수준(curl/bash_devtcp 등 여러 시나리오, connect 경로 포함)의 전체 반복 탐지율 측정은 kshield_vpatch_lsm에 대해 아직 없다.
- **동등성 검정(TOST) 수행:** 성능 무영향을 통계적으로 엄밀히 증명하려면 TOST 기반 동등성 검정과 표본 크기 확대($N \geq 30$)가 필요하다. 4.5절의 Falco/Tetragon 비교에는 이미 TOST(마진 $\pm 2\%$, 사전 고정)를 적용하였으나 $N=20 \cdot \text{VM} 1 \text{대} \cdot \text{세션} 1 \text{회}$ 에 그친다 — 여러 세션에 걸친 반복과 더 큰 표본으로 재검증할 필요가 있다.
- **Falco "무릎 시 오버헤드 증가" 기전 규명:** 4.5.1절의 대조군 실험(falco_norules/tetragon_norules)으로 Tetragon의 오버헤드는 정책 내용과 무관한 에이전트 기반 비용이며, kShield의 경량성이 AI 서버 전용으로 좁게 쓴 룰 덕분이 아님을 확인했다. 다만 Falco에서 룰을 제거했을 때 오버헤드가 오히려 커진 현상에 대해 제시한 "이벤트 유형 필터링" 설명은 libsinsp 내부 계측으로 검증하지 않은 추정이다 — 코드 수준 확인이나 이벤트 유형별 더미 룰 같은 후속 대조군으로 확정할 필요가 있다.

적용 범위 확장(우선순위 낮음)

- **다른 CVE로의 일반화:** 4.6절에서 TorchServe(CVE-2023-43654)·MLflow(CVE-2024-37054)·Triton(CVE-2023-31036)·vLLM(CVE-2025-66448)·BentoML(CVE-2024-2912/2025-27520)·Gradio(CVE-2024-1561)·text-generation-webui(CVE-2025-12487/88) 일곱 사례에 대해 커널 코드 변경 없이 kshield_ctl parent-add 런타임 등록만으로 탐지됨을 예비 확인하였고, 4.6.1절에서 같은 일곱 사례에 대해 Falco·Tetragon도 룰/정책에 이름·경로만 추가하면 동일하게 탐지함을 확인하였다.

다만 전부 1회 기능 확인에 그쳐, 4.3~4.5절 수준의 반복 통계 측정과 세 도구 간 오버헤드·성능 비교는 아직 없다. 일곱 프레임워크 모두 이 수준까지 검증을 넓힐 필요가 있으며, 아직 다루지 않은 다른 CVE·프레임워크로도 계속 확장할 수 있다.

- **실제 쿠버네티스 네임스페이스 인지:** cgroup ID를 실제 K8s 네임스페이스로 매핑하려면 K8s API 서버를 감시하는 별도 컨트롤 플레인 이 필요하다.
- **syslog 연동의 종단 검증:** 실제 SIEM까지 도달·파싱되는지는 검증하지 않았다.
- **kshield_ctl 대량 설정 지원:** 여러 규칙을 한 번에 넣는 설정 파일/벌크 가져오기 기능은 아직 없다.
- **고동시성 환경에서의 시스템 전역 오버헤드:** 다수의 무관한 프로세스가 동시다발적으로 connect()를 시도하는 상황은 아직 측정하지 않았다.
- **정밀 per-fork 비용의 여러 세션 재검증:** 4.4절에서 bpf_ktime_get_ns() 기반 격리 측정으로 fork+exit 혹은 자체의 비용이 약 1.72마이크로초임을 확인하여, 기존 end-to-end 추정치(약 0.2마이크로초)가 실제보다 8~9배 낮게 잡혀 있었음을 밝혔다. 다만 이 정밀 측정도 단일 세션 1회에 그치므로, 여러 세션에 걸친 반복 측정으로 재확인할 필요가 있다.
- **실제 Ray의 fork 패턴 재현:** 실제 Ray 클러스터의 태스크 처리량·fork 빈도를 프로파일링하여 더 현실적인 워크로드로 재측정할 필요가 있다.
- **IPv6 신뢰 목적지 목록 지원:** 현재 IPv6는 loopback 외 모든 목적지를 의심으로 간주한다.
- **DNS 터널링 등 잔여 우회 대응:** SHADOW_CONNECT로도 막지 못하는 우회에 대한 추가 탐지 계층 연구가 필요하다.
- **프로세스명 위장 및 인프로세스 코드 실행 대응:** watch_self[]는 comm이 정확히 일치하는 경우만 다룬다.
- **재시작 핸드오프 순간의 마이크로초 단위 정밀 계측:** 3.9절에서 밀리초 단위(150회, 약 15~20ms 간격, 단일 세션 1회, kshield_vpatch·kshield_vpatch_lsm 두 컴포넌트 모두)로는 재시작 핸드오프 구간에 공백이 없음을 실측으로 확인하였다. 다만 이는 유저스페이스 프로세스(nc) fork/exec 기반 측정의 해상도 한계 안에서의 결과이며, 코드상 진짜 위험 구간인 bpf_link__pin/unlink 교체 찰나의 마이크로초 단위 공백 유무는 bpf_ktime_get_ns() 기반의 커널 내부 계측과 여러 세션에 걸친 반복 측정이 필요하다.

- **watched_self[]의 execveat() 기반 self-exec 치환**
참지: 3.10절(6차)에서 execve() 기반 self-exec 치환 (bash의 단일 명령 exec 최적화)은 새 혹은 (tp/syscalls/sys_enter_execve)으로 해소하였다. 다만 execveat()(예: fexecve())를 통한 동일한 치환은 별도 혹은 없어 여전히 놓칠 수 있으며, 이에 대한 보완과 N회 반복 검증이 필요하다.

참고문헌

- [1] Oligo Security, "ShadowRay: First Known Attack Campaign Targeting AI Workloads Exploited In The Wild," Mar. 2024. [Online]. Available: <https://www.oligo.security/blog/shadowray-attack-ai-workloads-actively-exploited-in-the-wild>
- [2] Anyscale, "Update on Ray CVEs CVE-2023-6019, CVE-2023-6020, CVE-2023-6021, CVE-2023-48022, CVE-2023-48023," Anyscale Blog, Nov. 2023. [Online]. Available: <https://www.anyscale.com/blog/update-on-ray-cves-cve-2023-6019-cve-2023-6020-cve-2023-6021-cve-2023-48022-cve-2023-48023>
- [3] MITRE/NVD, "CVE-2023-48022," National Vulnerability Database, 2023. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2023-48022>
- [4] GitHub Advisory Database, "Ray has arbitrary code execution via jobs submission API (GHSA-6wgj-66m2-xxp2)," 2023. [Online]. Available: <https://github.com/advisories/GHSA-6wgj-66m2-xxp2>
- [5] Falco, "Container Runtime Security," The Falco Authors, CNCF, 2024. [Online]. Available: <https://falco.org>
- [6] Isovalent, "Tetragon: eBPF-based Security Observability and Runtime Enforcement," GitHub, 2023. [Online]. Available: <https://github.com/cilium/tetragon>
- [7] OWASP Foundation, "Virtual Patching Best Practices," 2024. [Online]. Available: https://owasp.org/www-community/Virtual_Patching_Best_Practices
- [8] 이상현, "kShield: eBPF 기반 AI 모델 보안 프레임워크 구현 코드," GitHub, 2026. [Online]. Available: <https://github.com/sangheon-lee1028/Ai-ebpf>
- [9] 이상현, "kShield-VirtualPatch: eBPF 기반 AI 서빙 프레임워크 취약점 가상 패치 구현 코드," GitHub, 2026. [Online]. Available: <https://github.com/sangheon-lee1028/kShield-VirtualPatch>
- [10] Oligo Security, "ShadowRay 2.0: Active Global Campaign Hijacks Ray AI Infrastructure Into Self-Propagating Botnet," Nov. 2025. [Online]. Available: <https://www.oligo.security/blog/shadowray-2-0-attackers-turn-ai-against-itself-in-global-campaign-that-hijacks-ai-into-self-propagating-botnet>
- [11] Anyscale, "CVE-2025-62593 and the CISA KEV listing: what Ray users need to know," Anyscale Blog, Aug. 2026. [Online]. Available: <https://www.anyscale.com/blog/ray-cve-2025-62593-kev-what-you-need-to-know>
- [12] Oligo Security, "ShellTorch: Multiple Critical Vulnerabilities in PyTorch Model Server (TorchServe) (CVSS 9.9, CVSS 9.8)," Oct. 2023. [Online]. Available: <https://www.oligo.security/blog/shelltorch-torchserve-ssrf-vulnerability-cve-2023-43654>
- [13] Snyk, "Deserialization of Untrusted Data in mlflow (CVE-2024-37054)," Snyk Vulnerability Database, 2024. [Online]. Available: <https://security.snyk.io/vuln/SNYK-PYTHON-MLFLOW-7210300>
- [14] NVIDIA, "Security Bulletin: NVIDIA Triton Inference Server - December 2023 (CVE-2023-31036)," 2023. [Online]. Available: <https://nvidia.custhelp.com/app/answers/detail/bulletin:-nvidia-triton-inference-server---december-2023>
- [15] ZeroPath, "vLLM Remote Code Execution via Model Config Auto-Mapping: CVE-2025-66448 Brief Summary," 2025. [Online]. Available: <https://zeropath.io/cve-2025-66448>

- <https://zeropath.com/blog/cve-2025-66448-vllm-rce-automap>
- [16] BentoML, "Remote Code Execution (RCE) Caused by Insecure Deserialization (GHSA-33xw-247w-6hmc, CVE-2024-2912/CVE-2025-27520)," GitHub Security Advisory, 2025. [Online]. Available: <https://github.com/bentoml/BentoML/security/advisories/GHSA-33xw-247w-6hmc>
- [17] Gradio, "gradio vulnerable to arbitrary method invocation via /component_server (GHSA-g9cj-cfpp-4g2x, CVE-2024-1561)," GitHub Advisory Database, 2024. [Online]. Available: <https://github.com/advisories/GHSA-g9cj-cfpp-4g2x>
- [18] SentinelOne, "CVE-2025-12487: Text-Generation-WebUI RCE Vulnerability," SentinelOne Vulnerability Database, 2025. [Online]. Available: <https://www.sentinelone.com/vulnerability-database/cve-2025-12487/>